

INFORME TÉCNICO INTEGRAL

Equipo 27

Julian David Miranda Leguizamón

Sebastian Bedoya Florez

Edgar Andres Romero Otalora

Jhon Alexander Tenjo Romero

Universidad de la Sabana

Maestría en Arquitectura de Software

Miguel Alfonso Varela Fonseca

Bogotá D.C.

Índice

Resumen Ejecutivo	5
Contexto del proyecto Ecommify	5
Arquitectura híbrida implementada	6
Motor transaccional - PostgreSQL	6
Motor analítico - MongoDB	6
Mecanismo de Sincronización	7
Principales logros y métricas de éxito	8
En PostgreSQL (Write Model)	8
En MongoDB (Read Model)	9
Recomendaciones estratégicas clave	9
Introducción y contexto	10
Problema de negocio	10
Objetivos y alcance	12
Objetivo general	12
Objetivos específicos	12
Alcance	13
Metodología aplicada	14
Análisis de Requerimientos y Diseño Conceptual	14
Modelado de Datos Políglota e Implementación de Patrones	14
Optimización Basada en Evidencia y Diagnóstico	15
Diseño de Sincronización e Integración Asíncrona	15
Planificación de Alta Disponibilidad y Escalabilidad Lineal	15
Arquitectura y diseño	16
Diagrama de arquitectura híbrida completa	16
Matriz de decisiones	17
PostgreSQL	17
MongoDB	18
Modelos ER	19
Modelo de documentos	20
Products	20
Order reviews	20
Geolocation	21
Implementación técnica	21
Resumen de implementación PostgreSQL	21
Tipos de datos utilizados	21
UUID	21
Varchar	21
Int2 (Small int)	21
Int8 (Integer)	22

Numeric	22
Jsonb	22
Timestamp	22
Extensiones	22
UUID-OSSP	22
PG_TRGM	22
Estrategia de indexación	23
Particionamiento	23
Resumen de implementación MongoDB	24
Estrategia de Modelado y Patrones Documentales	24
Estrategia de Indexación	25
Procesamiento y Pipelines de Agregación	25
Enfoque en decisiones técnicas NO obvias y sus justificaciones	26
Evaluación de rendimiento y escalabilidad	29
Metodología de pruebas aplicada	29
Resultados de pruebas de carga y escalabilidad	30
Identificación de cuellos de botella y limitaciones encontradas	32
Gráficas y métricas clave	32
MongoDB	33
Metodología de pruebas aplicada	33
Resultados de pruebas de carga y escalabilidad	34
Escalabilidad (geolocation particionada):	34
Análisis comparativo PostgreSQL vs MongoDB	35
Identificación de cuellos de botella y limitaciones encontradas	36
Gráficas y métricas clave	37
Análisis arquitectónico crítico	38
Trade-offs del Teorema CAP	38
MongoDB	40
Products	41
Order_reviews	41
Geolocation	42
Evaluación de consistencia eventual: ventanas de inconsistencia y estrategias de mitigación	43
PostgreSQL	43
MongoDB	45
Origen de la ventana de inconsistencia	45
Evaluación por módulo	46
Estrategias de mitigación implementadas	47
Síntesis crítica	47
Reflexión: ¿fue correcta la selección de tecnología por módulo? ¿Qué cambiaría?	48

PostgreSQL	48
MongoDB	49
Products	49
Order_reviews	50
Geolocation	50
Conclusión transversal	51
Lecciones aprendidas sobre arquitecturas híbridas	51
Recomendaciones estratégicas	53
Plan de escalamiento para crecimiento 10x	53
Sharding geográfico (geolocation)	55
Particionamiento (order_items)	55
Read réplica para desacoplar cargas	55
Fases de escalamiento	56
Migración planificada de Free Tier a Producción	56
Limitaciones del entorno actual	56
Dimensionamiento y costos estimados	57
Tecnologías complementarias recomendadas	57
Caché distribuido - Redis	57
Motor de búsqueda - Elasticsearch / OpenSearch	58
Observabilidad - Prometheus y Grafana	59
Estrategia CI/CD para cambios de esquema	61
Versionamiento de esquema en PostgreSQL – Liquibase / Flyway	61
Validación de esquema en MongoDB – \$jsonSchema	61
Pipeline de CI/CD con GitHub Actions	62
Conclusiones	63
Cumplimiento de objetivos	63
Objetivo 1 — Esquema relacional normalizado en PostgreSQL.	63
Objetivo 2 — Esquema documental en MongoDB para lectura masiva.	63
Objetivo 3 — Decisiones arquitectónicas bajo el Teorema CAP.	64
Objetivo 4 — Recomendaciones estratégicas de escalamiento.	64
El objetivo general -Diseñar, implementar y evaluar críticamente la arquitectura políglota híbrida-	65
Principales aprendizajes del equipo	65
Reflexión final sobre el proceso	67

Resumen Ejecutivo

Contexto del proyecto Ecommify

Ecommify es una empresa de comercio electrónico enfocada en conectar vendedores y compradores dentro de una plataforma digital centralizada. La solución permitirá a diferentes comercios ofrecer productos en línea, gestionar órdenes de compra, procesar pagos y coordinar entregas a clientes ubicados en distintas regiones.

Para dar soporte a la operación global del marketplace, el negocio se estructura en torno a múltiples procesos core interdependientes:

- **Gestión de Catálogos:** Registro, categorización y actualización de la oferta comercial de los vendedores.
- **Ciclo de Órdenes y Pagos:** Procesamiento de compras, validación de transacciones financieras y pasarelas de pago electrónico.
- **Logística y Distribución:** Seguimiento de pedidos, coordinación de despachos y administración de entregas en diversas regiones.
- **Interacción y Reputación:** Registro de reseñas y calificaciones por parte de los clientes para establecer métricas de confianza dentro de la comunidad.

Además de la operación diaria, Ecommify contempla una dimensión analítica e interactiva crítica basada en la localización. El negocio requiere procesar datos geográficos masivos para identificar la distribución de la demanda, mapear la ubicación de los vendedores, optimizar los tiempos de despacho logístico y detectar patrones de consumo regionales que guíen las decisiones estratégicas de la compañía. En consecuencia, el modelo operativo de Ecommify demanda la capacidad de soportar un flujo constante de transacciones simultáneas de alta

fidelidad, junto con una intensa actividad de consulta de datos para la generación de reportes e indicadores de negocio en tiempo real.

Arquitectura híbrida implementada

Para resolver la correcta operabilidad entre las transacciones y las consultas analíticas masivas, Ecommify implementa una estrategia de persistencia híbrida. Esta arquitectura logra desacoplar el modelo de datos según sus requerimientos de acceso y almacenamiento a través de dos motores coordinados por un bus de eventos.

Motor transaccional - PostgreSQL

PostgreSQL actúa como la verdad absoluta transaccional para los procesos del core de negocio cumpliendo con las propiedades ACID.

- Dominio Asignado: Gestión del ciclo de vida de órdenes de compra, agregación de ítems, procesamiento de pagos y control de inventario.
- Justificación Técnica: Garantiza la integridad referencial y evita anomalías operativas (como dobles cobros o sobreventa de stock) mediante el uso de restricciones relacionales, transacciones complejas y bloqueos optimistas/pesimistas según el flujo.

Motor analítico - MongoDB

MongoDB se utiliza como la capa de operaciones de lectura intensiva y persistencia de estructuras de datos variables.

- **Dominio Asignado:** Catálogo de productos dinámico, persistencia e historial de reseñas/calificaciones, y el almacenamiento optimizado de datos geográficos para análisis regional.
- **Optimización y Patrones Implementados:** Para asegurar tiempos de respuesta en el orden de los milisegundos, se implementan patrones avanzados de modelado documental:
 - **Extended Reference Pattern:** Duplicación controlada de datos del vendedor y producto en las órdenes de lectura para evitar joins en tiempo de ejecución.
 - **Computed Pattern:** Pre Cómputo de agregaciones (como el promedio de calificaciones de productos) para mitigar el impacto en CPU durante consultas masivas.
 - **Optimización Geoespacial:** Desacoplamiento y deduplicación masiva de la información de geolocalización, consolidando millones de coordenadas dispersas en un conjunto altamente eficiente de registros únicos indexados geoespacialmente de forma nativa (2dsphere).

Mecanismo de Sincronización

La sincronización de datos entre el almacenamiento transaccional y el analítico se realiza de manera asíncrona a través de una arquitectura guiada por eventos utilizando **Apache Kafka** como Message Broker.

- **Garantía de Consistencia Eventual:** Dado que los motores están desacoplados, el sistema opera bajo un modelo de consistencia eventual, donde los cambios en la capa transaccional se reflejan en la capa de lectura en cuestión de milisegundos.

- **Patrón Outbox Transaccional:** Para evitar fallos de consistencia dual (escribir en la base de datos pero fallar al publicar en el broker), las escrituras en PostgreSQL incluye de forma atómica la inserción del evento en una tabla Outbox dentro de la misma transacción. Un componente especializado lee esta tabla y publica los mensajes en los tópicos correspondientes de Kafka.
- **Resiliencia y Tolerancia a Fallos:** Los consumidores encargados de actualizar MongoDB están diseñados bajo el principio de idempotencia (procesar el mismo mensaje múltiples veces produce el mismo resultado) y cuentan con soporte de colas de descarte (Dead Letter Queues - DLQ) para aislar y procesar eventos malformados sin detener el flujo de sincronización general.

Principales logros y métricas de éxito

La implementación de la arquitectura políglota híbrida para Ecommify alcanzó mejoras de rendimiento medibles y consistentes en ambos motores de base de datos, validando las decisiones de diseño tomadas en la fase conceptual.

En PostgreSQL (Write Model)

- La indexación GIN con pg_trgm sobre campos de búsqueda textual (ciudades, categorías) redujo los tiempos de respuesta entre 50% y 98%, según el entorno de validación (Supabase vs Colab).
- La indexación B-tree sobre llaves foráneas mejoró el historial de compras por cliente entre 20% y 72.5%, eliminando hash joins costosos.
- Las consultas de agregación sobre estados poco frecuentes (ej. canceled) mejoraron hasta 60%, mientras que el optimizador de costos de PostgreSQL demostró

comportamiento inteligente: para estados de alta selectividad como delivered, mantuvo el escaneo secuencial por ser más eficiente que un acceso indexado — evidencia de que el motor toma decisiones basadas en costo real, no en la sola existencia de índices.

En MongoDB (Read Model)

Colección	Docs examinados (antes → después)	Tiempo de ejecución	Mejora
Products	65,902 → 27,356	183 ms → 156 ms	↓ 14.8%
Order Reviews	198,448 → 75,917	372 ms → 113 ms	↓ 69.6%
Geolocation	2,000,326 → 6,349	372 ms → 113 ms	↓ 99.4% (162x)

El logro más significativo del proyecto es la optimización de geolocation: la combinación de rediseño de esquema (deduplicación por prefijo postal) con un índice compuesto state-city produjo un factor de aceleración de 162 veces, con un Index Efficiency Ratio de 1.00 (cero falsos positivos) en las tres colecciones optimizadas.

Recomendaciones estratégicas clave

A partir de los resultados de rendimiento obtenidos y de las proyecciones de crecimiento del marketplace, se identifican cuatro líneas estratégicas prioritarias para la evolución de la arquitectura de Ecommify:

1. Escalamiento horizontal anticipado para crecimiento 10x. Las colecciones con mayor tasa de crecimiento proyectado (geolocation, order_items) deben activar el diseño de Sharding ya definido antes de alcanzar los umbrales de degradación observados en la línea base (ej. los 2,000,326 documentos sin deduplicar en geolocation). Se

recomienda definir disparadores de capacidad (volumen y latencia) que activen la incorporación de nuevos shards de forma proactiva, no reactiva.

2. Migración planificada de free tier a producción. El entorno actual (Supabase/Colab para PostgreSQL, instancias locales para MongoDB) es adecuado para validación, pero no para carga productiva real. La migración debe considerar: dimensionamiento de cómputo y almacenamiento bajo demanda proyectada, costos de replicación (Replica Sets de 3 nodos por segmento) y el impacto económico de mantener sincronización vía Kafka en un entorno administrado.
3. Incorporación de tecnologías complementarias. Se recomienda un caché distribuido (Redis) para absorber las lecturas más frecuentes del catálogo y reducir presión sobre MongoDB; un motor de búsqueda dedicado (Elasticsearch/OpenSearch) para sustituir progresivamente las búsquedas difusas vía pg_trgm a medida que crezca el catálogo de productos; y una capa de observabilidad (Prometheus/Grafana o equivalente) que monitoree en tiempo real Index Efficiency Ratio, replication lag y throughput, dado que estas métricas fueron clave para justificar las optimizaciones ya implementadas.
4. Estrategia de CI/CD para cambios de esquema. La naturaleza híbrida del sistema exige versionado controlado en ambos motores: migraciones versionadas y reversibles en PostgreSQL (ej. Liquibase/Flyway) y validación de esquemas JSON (\$jsonSchema) en MongoDB integrada al pipeline de despliegue, evitando que un cambio de esquema rompa la sincronización Outbox/Kafka entre el Write Model y el Read Model.

Introducción y contexto

Problema de negocio

Ecommify es una plataforma de comercio electrónico que conecta vendedores y compradores en un marketplace centralizado, gestionando el ciclo completo de la operación comercial: catálogo de productos, procesamiento de órdenes, pagos, logística de entrega y experiencia post-venta a través de calificaciones y reseñas.

El crecimiento del marketplace expone una tensión estructural entre dos tipos de carga de trabajo con requisitos opuestos:

Carga transaccional crítica. Las operaciones de creación de órdenes, confirmación de pagos y gestión de clientes y vendedores exigen integridad referencial estricta y consistencia ACID.

Un fallo de consistencia en esta capa —por ejemplo, una orden registrada sin su pago asociado, o un pago duplicado— representa pérdida financiera directa y riesgo de incumplimiento ante auditorías. El análisis exploratorio del dataset Olist confirma la magnitud de esta carga: ~99,441 órdenes y ~103,886 registros de pago, con relaciones transaccionales que deben mantenerse íntegras incluso bajo escritura concurrente.

Carga analítica y de consulta masiva. En paralelo, la plataforma debe sostener tráfico de lectura mucho más voluminoso y volátil: navegación de catálogo, visualización de reseñas y cálculo de costos de envío por geolocalización en tiempo real durante el checkout. Esta carga no tolera la misma rigidez transaccional —de hecho, exige lo contrario: alta disponibilidad y baja latencia, incluso al costo de una ventana breve de inconsistencia. La colección de geolocalización por sí sola supera el millón de registros con crecimiento muy alto proyectado, un volumen que un modelo puramente relacional normalizado no sostendría con el rendimiento de respuesta en milisegundos que requiere el cálculo de envío en el carrito de compra.

Un único motor de base de datos no puede satisfacer ambas exigencias de forma óptima: forzar todo el catálogo y las reseñas a un esquema relacional normalizado introduciría JOINS costosos y cuellos de botella en lectura masiva; forzar las transacciones financieras a un modelo orientado a disponibilidad (NoSQL puro) comprometería la integridad contable que el negocio no puede arriesgar.

Este problema de negocio —sostener consistencia transaccional crítica y disponibilidad de lectura masiva de forma simultánea, sin que una carga degrade a la otra— es el que motiva la decisión arquitectónica central del proyecto: una arquitectura políglota híbrida, en la que PostgreSQL opera como Write Model transaccional y MongoDB como Read Model desacoplado, sincronizados de forma asíncrona orientada a eventos.

Objetivos y alcance

Objetivo general

Diseñar, implementar y evaluar críticamente una arquitectura de base de datos políglota híbrida (PostgreSQL + MongoDB) para la plataforma Ecommify, que garantice consistencia transaccional en las operaciones críticas del negocio y disponibilidad de alto rendimiento en las cargas de consulta masiva, con base en evidencia empírica de rendimiento y en los principios del Teorema CAP.

Objetivos específicos

1. Diseñar un esquema relacional normalizado en PostgreSQL para las entidades transaccionales del negocio (clientes, órdenes, pagos, vendedores), garantizando integridad referencial y propiedades ACID.

2. Diseñar un esquema documental en MongoDB para las entidades orientadas a lectura masiva (catálogo de productos, reseñas, geolocalización), aplicando patrones de modelado (Atributos, Cómputo, Polimórfico) que optimicen el acceso concurrente.
3. Documentar las decisiones arquitectónicas bajo el marco del Teorema CAP, justificando los trade-offs de consistencia y disponibilidad aceptados por módulo y por escenario operacional de negocio.
4. Formular recomendaciones estratégicas de escalamiento y migración a producción que permitan sostener el crecimiento proyectado del marketplace.

Alcance

El proyecto cubre el ciclo completo desde el diseño conceptual hasta la evaluación crítica de la implementación, utilizando como base el dataset público Olist Brazilian E-commerce (~100,000 registros transaccionales y más de 1,000,000 de registros geográficos) como proxy representativo de la operación de Ecommify.

Se incluye:

- Diseño conceptual y lógico de ambos modelos de datos (relacional y documental).
- Implementación y optimización medible en entornos de validación (PostgreSQL local/Supabase/Colab; MongoDB local con Replica Sets simulados).
- Diseño (no despliegue productivo) de Sharding para MongoDB.
- Evaluación de rendimiento mediante pruebas de queries y, en esta etapa final, pruebas de carga con usuarios concurrentes.

Queda fuera del alcance: el despliegue en infraestructura productiva real, la implementación de las tecnologías complementarias recomendadas (caching, búsqueda, observabilidad) y la

migración efectiva de free tier a producción —estos quedan formulados como recomendaciones estratégicas para una fase posterior del proyecto.

Metodología aplicada

El desarrollo del proyecto se rigió por una metodología iterativa dividida en cinco fases secuenciales. Este enfoque garantizó que cada decisión de diseño técnico estuviera respaldada por requerimientos operativos y validada mediante pruebas de rendimiento.

Análisis de Requerimientos y Diseño Conceptual

- Se identificaron los límites de contexto para separar las operaciones transaccionales de aquellas con alta carga de lectura y analítica.
- Se definió la estrategia de persistencia, asignando a **PostgreSQL** las responsabilidades del modelo de escritura ACID y a **MongoDB Atlas** las del modelo de lectura extendido.

Modelado de Datos Políglota e Implementación de Patrones

- **Modelo Relacional:** Se normalizó el esquema en PostgreSQL para asegurar la integridad referencial de órdenes, ítems y pagos.
- **Modelo Documental:** En MongoDB se rompió deliberadamente la normalización mediante la aplicación de patrones de diseño avanzados (Extended Reference, Computed Pattern y Esquemas Polimórficos) para consolidar en documentos únicos la información requerida por la interfaz de usuario.

Optimización Basada en Evidencia y Diagnóstico

- **Análisis de Planes de Ejecución:** Se ejecutaron pruebas de carga simuladas y se analizaron los planes de ejecución mediante la herramienta explain de MongoDB para identificar cuellos de botella en consultas críticas de productos, reseñas y datos geoespaciales.
- **Estrategia de Indexación:** A partir de los resultados del diagnóstico, se diseñaron e implementaron índices compuestos y geoespaciales nativos.
- **Deduplicación de Datos:** Se aplicó un proceso de refactorización y limpieza sobre conjuntos de datos masivos.

Diseño de Sincronización e Integración Asíncrona

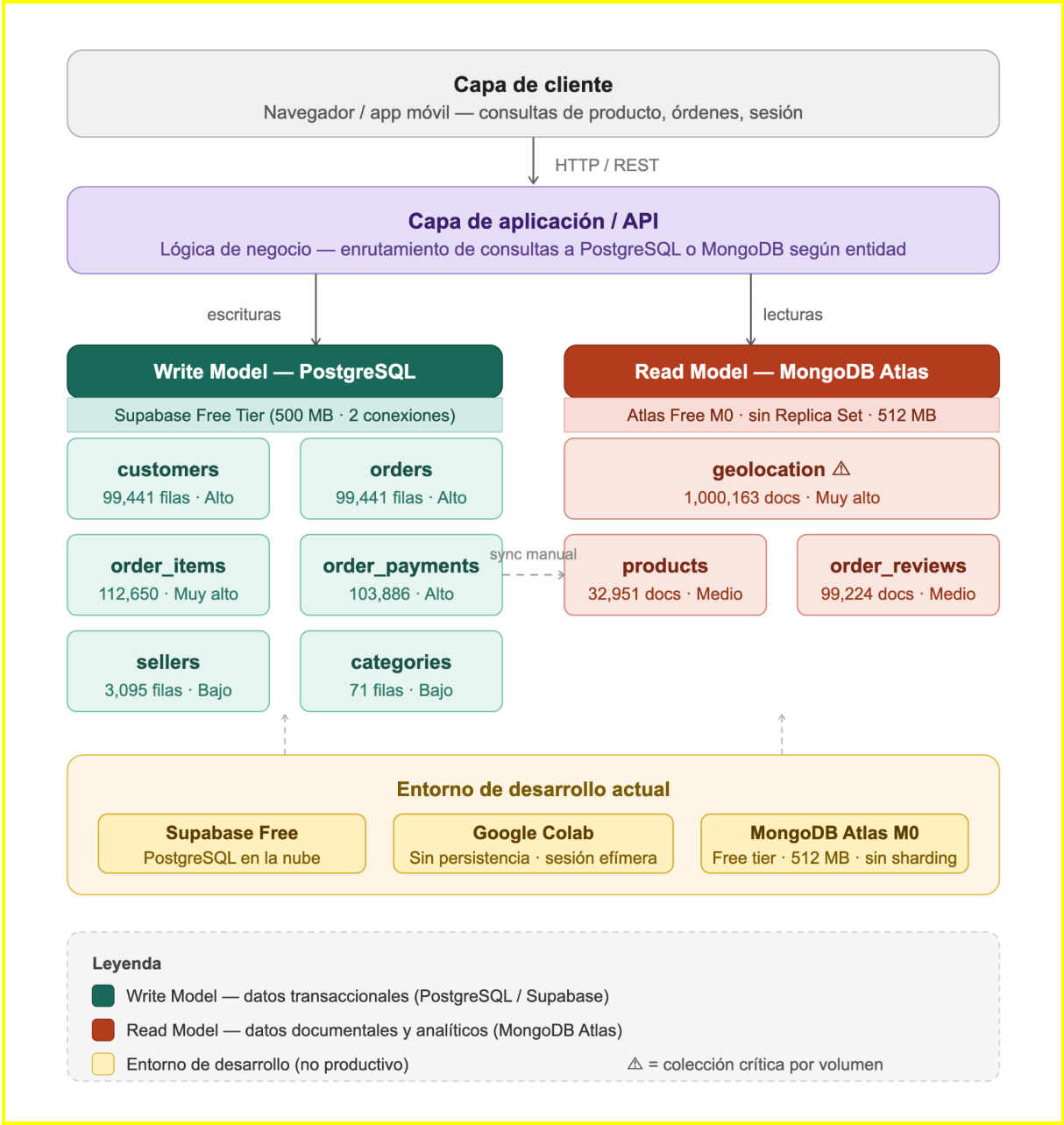
- Se modelaron los contratos de los eventos de dominio en formato JSON.
- Se estructuró la topología de tópicos en Apache Kafka.
- Se implementó a nivel de diseño el patrón Outbox Transaccional para garantizar la entrega atómica de mensajes.

Planificación de Alta Disponibilidad y Escalabilidad Lineal

- **Tolerancia a Fallos:** Se diseñó teóricamente una arquitectura de disponibilidad continua basada en Replica Sets de MongoDB para asegurar la conmutación por error automática.
- **Escalabilidad Horizontal:** Se definió la estrategia de distribución de datos mediante Sharding, seleccionando una clave de fragmentación dispersa (Hashed Shard Key) para garantizar una distribución uniforme de las escrituras y lecturas, evitando la creación de puntos calientes (hotspots) en el sistema.

Arquitectura y diseño

Diagrama de arquitectura híbrida completa



Matriz de decisiones

PostgreSQL

Colección	Estructura	Volumen y Velocidad	Teorema CAP
customers	Estructura altamente relacional. Contiene identificadores únicos de clientes, ciudad, estado y código postal. Relacionada directamente con orders.	Volumen medio (~99k registros). Velocidad media de lectura y escritura debido a consultas frecuentes de clientes y ubicación.	(C) PostgreSQL prioriza consistencia para garantizar integridad de datos de clientes y relaciones con pedidos.
orders	Tabla central transaccional con múltiples relaciones hacia customers, payments y order_items. Maneja estados y fechas del ciclo de vida del pedido.	Alto volumen (~99k pedidos) y alta velocidad transaccional. Escrituras frecuentes y consultas constantes.	(C) La consistencia es crítica para evitar pérdida o duplicidad de pedidos durante transacciones.
order_items	Estructura relacional detallada por producto vendido. Relaciona órdenes, productos y vendedores.	Alto volumen (~112k registros). Alta velocidad de inserción y consultas analíticas sobre ventas.	(C) Debe garantizar integridad entre pedidos, productos y vendedores.
order_payments	Datos financieros estructurados asociados a órdenes. Incluye cuotas, tipo de pago y montos.	Volumen alto (~103k registros). Escrituras moderadas y lecturas frecuentes para análisis financiero.	(C) Se prioriza consistencia para asegurar exactitud de pagos y facturación.
sellers	Información estructurada de vendedores con ubicación geográfica. Relacionada con order_items.	Volumen bajo (~3k vendedores). Lecturas frecuentes y pocas actualizaciones.	(C) Se requiere consistencia para mantener referencias correctas en ventas y logística.

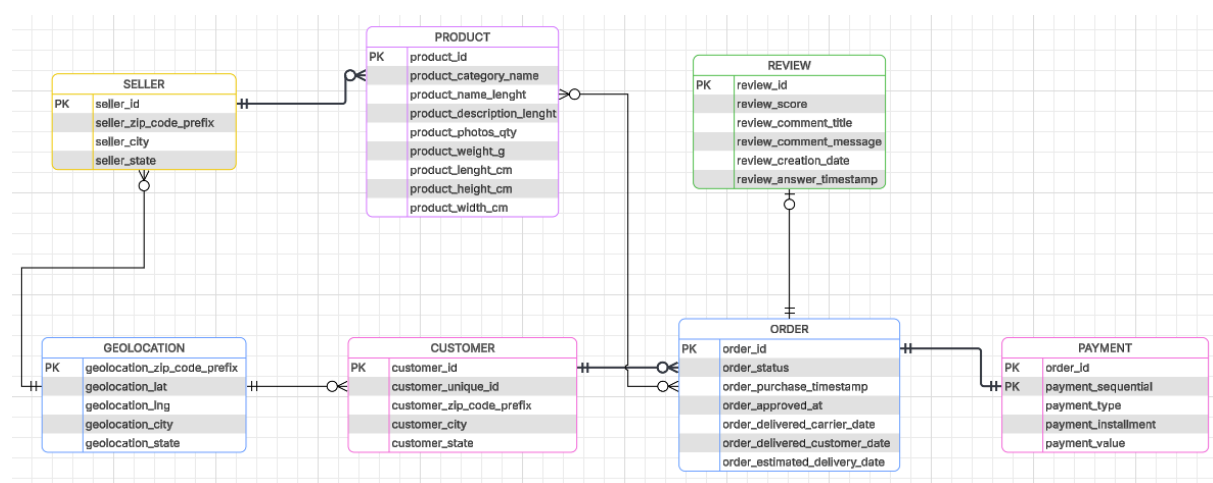
Colección	Estructura	Volumen y Velocidad	Teorema CAP
categories	Tabla pequeña de traducción de categorías entre portugués e inglés. Relación auxiliar con products.	Volumen muy bajo (71 registros). Baja frecuencia de acceso.	(CA) Puede mantenerse altamente disponible y consistente debido a su tamaño reducido y baja complejidad.

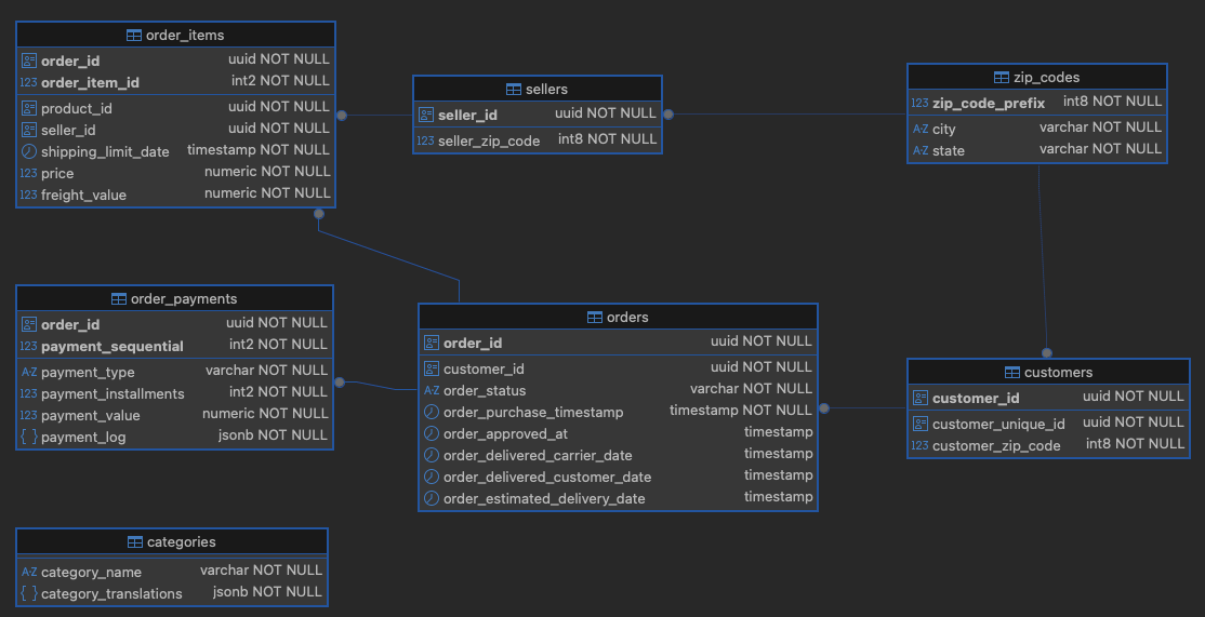
MongoDB

Colección	Estructura	Volumen y Velocidad	Teorema CAP
products	Estructura polimórfica: Los productos varían sus atributos según su categoría (ej: ropa utiliza tallas).	Volumen: Medio (~32k registros). Velocidad: Extremadamente Alta (Lecturas masivas concurrentes).	(P) El sistema está diseñado para sobrevivir a una división del clúster. (A) Ante dicha partición, el catálogo debe seguir respondiendo y mostrando los productos.
order_reviews	Estructura semi-estructurada: Documentos independientes vinculados por product_id. No todas las reseñas tienen la misma estructura.	Volumen: Alto (+100k registros y crece con cada orden). Velocidad: Media en Escritura y Alta en Lectura.	(P) Si la red se fragmenta entre centros de datos, los servidores de base de datos aislados siguen operando de forma autónoma. (A) Un cliente debe poder escribir o leer comentarios siempre. La caída de un enlace de red no puede bloquear la sección de opiniones.

geolocation	Estructura documental especializada (GeoJSON): Almacena puntos espaciales definidos por arreglos de coordenadas [longitud, latitud].	Volumen: Alto (Más de 1 millón de filas en el dataset original). Velocidad: Alta (Cálculos de proximidad en milisegundos para estimación de costo de envío durante el checkout).	(P) MongoDB maneja esta carga mediante Sharding. Si un nodo fragmentado regional sufre una desconexión, los demás shards siguen atendiendo consultas de sus respectivas regiones. (A) El cálculo del costo de envío en el carrito debe responder en milisegundos para evitar que el usuario abandone la compra.
-------------	--	--	---

Modelos ER





Modelo de documentos

Products

```
{
  "_id": "060cb19345d90064d1015407193c6ffc",
  "product_category_name": "perfumeria",
  "physical_specs": {
    "weight_g": 525,
    "length_cm": 17,
    "height_cm": 11,
    "width_cm": 12
  },
  "multimedia_metadata": {
    "product_name_length": 42,
    "product_description_length": 774,
    "photos_qty": 2
  },
  "performance_metrics": {
    "average_review_score": 4.6,
    "total_reviews_count": 18
  }
}
```

Order reviews

```
{
  "_id": "7ac549e1e29c625d3934329e43b546e5",
  "order_id": "e481f51cbdc54678b7349021f2804312",
  "review_score": 5,
  "review_comment": {
    "title": "Excelente servicio",
  }
}
```

```
    "message": "El producto llegó mucho antes de lo estimado y en perfectas condiciones."
  },
  "timestamps": {
    "creation_date": "2017-09-21T00:00:00Z",
    "answer_timestamp": "2017-09-22T10:57:03Z"
  }
}
```

Geolocation

```
{
  "_id": "01001",
  "geolocation_zip_code_prefix": 1001,
  "location": {
    "type": "Point",
    "coordinates": [-46.634007, -23.549807]
  },
  "city": "sao paulo",
  "state": "SP"
}
```

Implementación técnica

Resumen de implementación PostgreSQL

Tipos de datos utilizados

UUID

Se utiliza para identificadores únicos ya que garantiza unicidad global, evita cruces entre registros e incrementa la seguridad al no utilizar IDs secuenciales. Ejemplo: las llaves foráneas de las tablas ordenes, clientes, vendedores, entre otros.

Varchar

Se utiliza para almacenar cadenas de texto de longitud variable, permite definir límites máximos de longitud y es adecuado para datos textuales controlados. Ejemplo: estados de las órdenes, tipos de pagos, entre otros.

Int2 (Small int)

Se utiliza para almacenar valores pequeños y acotados como por ejemplo: número de cuotas, secuencia del pago, entre otros.

Int8 (Integer)

Se utiliza para almacenar valores enteros de mayor rango como por ejemplo el prefijo de los códigos postales.

Numeric

Se utiliza para datos de valores decimales ya que es fundamental para cálculos monetarios como por ejemplo: valor del pedido, precio de los productos, entre otros.

Jsonb

Se utiliza para almacenar información semiestructurada en formato json ya que permite realizar consultas eficientes. Ejemplo: campo para almacenar las traducciones de las categorías de los productos y el log de la transacción del pago del pedido.

Timestamp

Se utiliza para almacenar fecha y hora, como por ejemplo las fechas y horas de las diferentes etapas de la compra de un pedido hasta su entrega al cliente.

Extensiones

UUID-OSSP

Para usar el tipo de dato UUID y permitir que la base de datos genere automáticamente los id por si misma, es necesario activar la extensión uuid-oss. La extensión le permite a PostgreSQL tener un conjunto de funciones avanzadas, por ejemplo: `uuid_generate_v4()`. Esto permitirá que cuando se inserten registros en las tablas orders, customers, sellers, entre otros, se generará automáticamente un id único sin depender de que un lenguaje de programación lo deba realizar.

PG_TRGM

La implementación de índices GIN en las columnas city (zip_codes) y category_name (categories) hace necesario el uso de pg_trgm. La extensión divide las cadenas de texto en bloques de tres caracteres consecutivos (trigramas) para medir la similitud matemática entre

palabras. Esto permitirá que el sistema sea tolerante a errores ortográficos que puedan ser cometidos por los usuarios al momento de digitar su ciudad al momento de realizar sus pedidos, también permitirá optimizar las búsquedas de las categorías de los productos y de esta forma el sistema podrá ofrecer sugerencias automáticas al usuario cuando realiza búsquedas.

Estrategia de indexación

- Índices b-tree: Se definió para llaves foráneas de las diferentes tablas y campos para control operativo como el campo `order_status` y `payment_type`. Esto permite mejorar el rendimiento de búsquedas por igualdad, operaciones de ordenamiento y consultas por rangos, su eficiencia permite localizar registros de manera rápida.
- Índices GIN: Para optimizar las búsquedas parciales de texto se definieron índices tipo GIN. Con esto se busca optimizar las búsquedas realizadas en tablas como `zip_codes` y `categories` dejando de depender de escaneos completos de la tabla.

Particionamiento

Durante la fase de diseño de Ecommify, se evaluó la posibilidad de aplicar particionamiento de tablas, por ejemplo en `orders` o `order_items`, sin embargo, se determinó que no era necesario debido al volumen actual de datos, el cual puede gestionarse de manera eficiente con los índices actuales. Adicionalmente, el sistema realiza consultas analíticas que requieren acceder al histórico completo de información, por lo que el particionamiento podría incrementar la complejidad sin generar beneficios significativos de rendimiento. Por esta razón, se decidió mantener esta estrategia como una opción de escalabilidad para futuras etapas del proyecto cuando el crecimiento de datos lo justifique.

Resumen de implementación MongoDB

La capa de lectura, catálogo y analítica se implementó sobre MongoDB, aprovechando su flexibilidad esquemática y su capacidad para procesar grandes volúmenes de datos orientados a lecturas intensivas.

Estrategia de Modelado y Patrones Documentales

El diseño lógico se estructuró en torno a tres colecciones clave, aplicando patrones avanzados que respetan una política estricta de referenciación para balancear el rendimiento de lectura sin incurrir en redundancia de datos mutables.

- **Esquema Polimórfico:** Se aprovechó la flexibilidad del formato BSON para gestionar un catálogo con categorías heterogéneas. La colección almacena atributos variables en estructuras dinámicas, compartiendo una base común para el inventario.
- **Subdocumentos Embebidos de Dimensiones y Métricas:** Para evitar operaciones analíticas costosas en el modelo relacional, se estructuraron subdocumentos embebidos estáticos dentro de products:
 - **physical_specs:** Agrupa de forma nativa los atributos físicos invariables del producto (*weight_g, length_cm, height_cm, width_cm*).
 - **performance_metrics:** Almacena dimensiones analíticas precalculadas como *average_review_score*, permitiendo que el catálogo muestre la reputación del producto sin calcularla en tiempo real.
- **Referenciación Ligera:** Las reseñas se modelaron manteniendo un desacoplamiento limpio. El documento de la reseña conserva campos específicos del evento (*review_score, review_comment.title, review_comment.message*) y se vincula al

núcleo transaccional mediante el `order_id` como puente relacional, evitando la duplicación de datos transaccionales.

Estrategia de Indexación

La configuración de índices fue el resultado del diagnóstico y optimización de los canales de consulta mediante los Explain Plans de MongoDB.

- **Optimización de Atributos Físicos (*products*):** Se definieron índices estratégicos sobre las rutas de los subdocumentos empotrados como ***physical_specs.weight_g***. Permitiendo segmentar el catálogo por peso y dimensiones de forma eficiente.
- **Indexación por Rangos de Interacción (*order_reviews*):** Se implementó un índice directo sobre el campo ***review_score***. Al ejecutar filtros de satisfacción, el motor ejecuta un Index Range Scan acotado, minimizando el número de documentos examinados en memoria y reduciendo drásticamente los tiempos de procesamiento.
- **Indexación Compuesta de Localización (*geolocation*):** Se estructuró un índice compuesto basado en criterios de jerarquía geográfica. Esto garantiza que el motor resuelva las búsquedas de distribución regional directamente desde la estructura del índice, reduciendo la necesidad de lecturas secuenciales en disco.

Procesamiento y Pipelines de Agregación

El Aggregation Framework de MongoDB se utilizó como el motor de transformación para estructurar reportes operacionales y tableros de control, garantizando un procesamiento fluido de grandes volúmenes de datos.

- **Clasificación Analítica de Catálogo:** Mediante pipelines optimizados en la colección de ***products***, se implementaron etapas combinadas de ***\$match***, ***\$project*** y ***\$addFields***

con operadores condicionales. Permite categorizar dinámicamente los ítems y calcular promedios de peso y rendimiento analítico por categoría sin afectar la base transaccional.

- **Consolidación y Deduplicación Geoespacial:** A través de un pipeline de agregación enfocado en la depuración estructural, el universo de datos se consolidó en un subconjunto optimizado de 6,349 registros únicos de localización, eliminando por completo la redundancia.

Tras aplicar el índice compuesto sobre esta colección depurada, el Explain Plan de la consulta de agregación geográfica reflejó el escenario óptimo: el número de llaves examinadas coincide exactamente con los documentos devueltos, bajando los tiempos de respuesta analíticos a 11 milisegundos.

Enfoque en decisiones técnicas NO obvias y sus justificaciones

Más allá de la indexación estándar, el proyecto incluyó varias decisiones técnicas que contradicen la intuición convencional o que solo se validaron mediante evidencia empírica:

- El optimizador de PostgreSQL ignorando un índice disponible. Era esperable que crear un índice sobre `order_status` mejorará toda consulta sobre esa columna. Sin embargo, al filtrar por el valor `delivered` —que representa una proporción muy alta de la tabla `orders`— el planificador optó deliberadamente por un Seq Scan, ignorando el índice. Justificación: cuando una consulta recupera una fracción mayoritaria de la tabla, un Index Scan generaría más lecturas aleatorias a disco que un escaneo secuencial, encareciendo la operación. Esta decisión del motor —no del equipo— se documentó como evidencia de que la indexación debe medirse, no asumirse: el mismo índice sí mejoró hasta 60% las consultas sobre `canceled`, un valor de baja frecuencia.

- Rediseño de esquema en lugar de indexación como primera palanca de optimización (geolocation). La decisión técnica no obvia no fue qué índice crear, sino deduplicar antes de indexar. La colección cruda contenía 2,000,326 documentos porque el CSV original repite coordenadas por cada variación mínima dentro de un mismo código postal. Agrupar por `zip_code_prefix` y conservar una sola ocurrencia redujo la colección a 6,349 documentos antes de aplicar cualquier índice. Justificación: un índice compuesto sobre una colección mal diseñada tiene un techo de mejora limitado; el rediseño multiplicó el efecto del índice posterior, logrando el factor de aceleración de 162x.
- Se ubicó `physical_specs.weight_g` como campo líder del índice compuesto en `products`, no por ser el campo más selectivo estadísticamente, sino porque es el campo sobre el que recae el `$match` inicial del pipeline. Esto permite un `index range scan` en lugar de un `collection scan`, reduciendo Docs Examined de 65,902 a 27,356 (58.5% menos).

Corrección: el índice no es un `covering index`, como se afirmó originalmente. La evidencia lo desmiente: `Keys Examined = Docs Examined = 27,356`. Si cubriera realmente el pipeline, Docs Examined sería 0. Que ambos valores coincidan indica que MongoDB igual recuperó el documento completo por cada entrada de índice (`Index Scan + Fetch`), probablemente porque las etapas `$addFields` y `$group` no se resuelven directamente sobre las claves del índice y requieren el documento materializado.

Conclusión: el índice sí aporta valor (evita el `collection scan` inicial, explicando la baja de 185ms a 154ms), pero su beneficio se limita al filtrado (`$match`), no a una cobertura completa del pipeline.

- Uso de state como campo líder en geolocation pese a su baja cardinalidad (27 valores posibles en Brasil). La intuición estándar de modelado de índices ESR (Equality-Sort-Range) sugiere priorizar campos de alta cardinalidad. Justificación: dado que el filtro es de igualdad exacta (state = "SP") y la colección ya fue deduplicada, la baja cardinalidad no penaliza la selectividad real sobre el conjunto reducido; en cambio, mantener state como líder permite que el mismo índice sirva tanto al filtrado como al agrupamiento posterior por city, cubriendo dos operaciones del pipeline con una sola estructura.
- Referenciación ligera en lugar de embedding completo para las traducciones de categoría. Pese a que el patrón general en MongoDB favorece la desnormalización por embedding para evitar JOINS, el equipo decidió no incrustar el diccionario de traducciones de categorías (product_category_name_translation) dentro de cada documento de producto. Justificación: es un atributo transversal y de bajo volumen (71 registros) que cambiaría la estructura de miles de documentos ante cualquier actualización; se mantiene como string-puente (product_category_name) y se resuelve la traducción en la capa de servicios, evitando duplicidad masiva de un dato que no aporta valor analítico al documento individual.
- Decisión de NO particionar orders y order_items pese al alto volumen. Contraintuitivamente, un volumen de ~99k-112k registros con crecimiento "alto"/"muy alto" no se tradujo en partición de tablas. Justificación: el análisis con datos reales mostró que los índices existentes ya gestionaban eficientemente ese volumen, y que el particionamiento habría introducido complejidad operativa (gestión de particiones, posible degradación en consultas analíticas que requieren el histórico completo) sin beneficio medible. La decisión se documentó explícitamente como una no-decisión basada en evidencia, no como omisión.

- Patrón Outbox Transaccional en lugar de publicación directa de eventos. En vez de publicar el evento a Kafka inmediatamente después del commit (enfoque más simple e "intuitivo"), se optó por escribir el evento en una tabla Outbox dentro de la misma transacción ACID. Justificación: publicar directamente expone una ventana de fallo (el commit puede tener éxito y la publicación a Kafka fallar, o viceversa), generando pérdida o duplicidad de eventos. El patrón Outbox garantiza atomicidad entre el cambio de estado y el registro del evento a publicar, a costa de un componente adicional de lectura (relay) que se justifica por la criticidad de no perder eventos entre el Write Model y el Read Model.

Evaluación de rendimiento y escalabilidad

PostgreSQL

Metodología de pruebas aplicada

Para evaluar el rendimiento y la escalabilidad de PostgreSQL se realizaron pruebas de ejecución sobre consultas representativas del sistema utilizando Google Colab como entorno de pruebas y SQLAlchemy como mecanismo de conexión a la base de datos.

Se identificaron cinco consultas críticas (búsqueda de categorías, búsqueda de ciudades, historial de compras de un cliente, reporte de ventas totales por estado y reporte de ventas canceladas). Posteriormente se aplicaron optimizaciones orientadas a mejorar el rendimiento de las consultas mediante el uso de estrategias más eficientes de acceso a los datos.

Para analizar el comportamiento de las consultas se utilizó la instrucción EXPLAIN ANALYZE, obteniendo el plan de ejecución real antes y después de la optimización. Esto permitió identificar y comparar las principales operaciones del plan de ejecución, tales como

Seq Scan, Index Scan, Hash Join y Sort, así como medir el impacto de las optimizaciones sobre los tiempos de respuesta.

Adicionalmente, se realizaron pruebas de carga y escalabilidad con el fin de evaluar el comportamiento de la base de datos bajo diferentes niveles de concurrencia. Para ello se seleccionaron consultas representativas y se ejecutaron de manera simultánea mediante Python utilizando la librería `concurrent.futures.ThreadPoolExecutor`.

Se utilizó la consulta "**Historial de compras de un cliente**" porque fue la consulta con los tiempos más altos en el análisis realizado con EXPLAIN ANALYZE.

Las pruebas se hicieron con 1, 10, 20 y 30 usuarios y se querían hacer con 40 usuarios pero se identificó una limitación en el pool de conexiones que generó errores de timeout.

Resultados de pruebas de carga y escalabilidad

A continuación se visualiza la tabla comparando el tiempo mínimo, el tiempo máximo y el promedio de los diferentes niveles de concurrencia.

Usuarios concurrentes	Tiempo mínimo (s)	Tiempo promedio (s)	Tiempo máximo (s)
10 usuarios	16.187	17.147	17.971
20 usuarios	20.327	24.761	29.577
30 usuarios	17.404	28.997	38.337

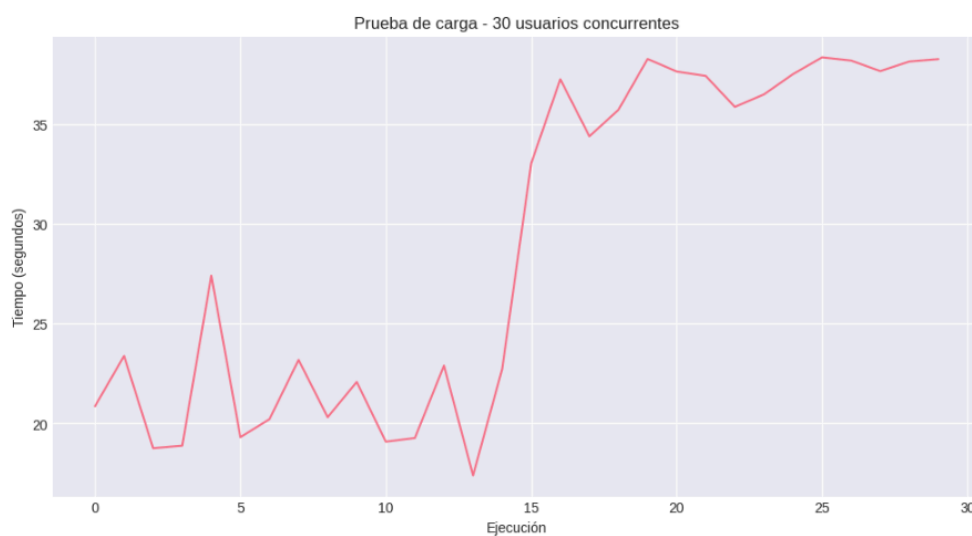
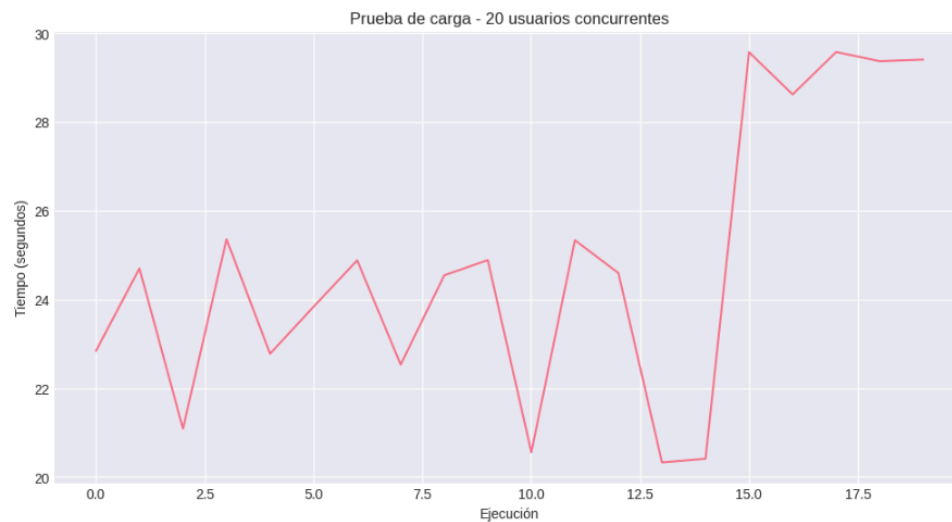
El tiempo para un usuario es de 1.4893 segundos.

Los resultados obtenidos evidencian una degradación progresiva del rendimiento de la base de datos a medida que aumenta la concurrencia.

Además, se observa un incremento en la variabilidad de los tiempos, con máximos que llegan hasta aproximadamente 38.33 segundos.

Este comportamiento indica que el sistema no escala de forma lineal, ya que el aumento en la cantidad de usuarios genera un crecimiento significativo en la latencia de las consultas.

A continuación se muestran las gráficas del comportamiento para 20 y 30 usuarios.



Identificación de cuellos de botella y limitaciones encontradas

Durante las pruebas de carga se identificaron cuellos de botella principalmente asociados al manejo de la concurrencia y al acceso simultáneo a la base de datos. A medida que aumentó el número de usuarios, se evidenció un incremento significativo en los tiempos de respuesta, lo que sugiere saturación en el uso de recursos del motor de base de datos y competencia por conexiones activas. Adicionalmente, se observó que la gestión del pool de conexiones de SQLAlchemy puede limitar el rendimiento bajo alta concurrencia, generando tiempos de espera o errores en escenarios de carga elevada. En conjunto, estos factores afectan la escalabilidad del sistema y provocan una degradación progresiva del rendimiento a medida que aumenta la carga de trabajo.

Gráficas y métricas clave

	Consulta	Plan de ejecución antes	Antes (ms)	Plan de ejecución despues	Después (ms)	Mejora (%)
0	Búsqueda de categorías	Seq Scan	5.473	Seq Scan	0.068	98.757537
1	Búsqueda de ciudades	Seq Scan	86.503	Bitmap Heap Scan y Bitmap Or	3.041	96.484515
2	Historial de compras de un cliente	Seq Scan y hash join	2307.889	Index Scan	1323.246	42.664227
3	Reporte de ventas totales	Seq Scan, hash join y sort	225.461	Seq Scan, hash join y sort	265.891	-17.932148
4	Reporte de ventas canceladas	Seq Scan, hash join y sort	12.978	Index Scan, Seq Scan, hash join y sort	6.231	51.987980



MongoDB

Metodología de pruebas aplicada

Se diseñaron dos suites de pruebas complementarias a las pruebas de consulta individual ya documentadas por colección:

1. **Pruebas de carga concurrente:** se simularon clientes simultáneos mediante `ThreadPoolExecutor`, ejecutando aleatoriamente una de las tres queries optimizadas (`products`, `order_reviews`, `geolocation`) en niveles crecientes de concurrencia: 1, 5, 10, 25 y 50 usuarios, con 3 peticiones por usuario.
2. **Pruebas de escalabilidad:** se ejecutó el pipeline optimizado de `geolocation` —ya particionado en 5 colecciones regionales (19,015 documentos totales tras deduplicación)— filtrando por 25%, 50%, 75% y 100% de los 27 estados de Brasil, midiendo `docsExamined` y `executionTimeMillis` reales vía `explain()`.

Resultados de pruebas de carga y escalabilidad

Carga concurrente:

Usuarios	Peticiones	Throughput (q/s)	Latencia prom. (ms)	Latencia P95 (ms)	Latencia máx. (ms)
1	3	18.30	54.32	45.55	79.89
5	15	71.55	63.55	155.78	158.94
10	30	144.23	60.09	170.99	196.64
25	75	50.29	299.36	1448.94	1479.69
50	150	75.14	457.27	1951.06	1976.45

Escalabilidad (geolocation particionada):

% estados	Nº estados	Tiempo total (ms)	Docs Examined	executionTime Millis
25%	6	49.90	19015	19
50%	13	51.27	19015	23
75%	20	48.99	19015	25
100%	27	51.02	19015	26

El throughput escala bien hasta 10 usuarios (144.23 q/s, el pico de toda la prueba), pero colapsa a 50.29 q/s en el salto a 25 usuarios —una caída del 65%— antes de recuperarse parcialmente en 50 usuarios sin volver al pico de 10. La escalabilidad sobre geolocation es estable y casi plana (19-26 ms) en todo el rango, lo cual es coherente con que docsExamined se mantiene constante en 19,015: el filtro por state ya está acotado a una sola partición física, así que aumentar el número de estados solicitados no aumenta proporcionalmente el trabajo del motor dentro del rango probado.

Análisis comparativo PostgreSQL vs MongoDB

Para justificar de forma cuantitativa y cualitativa la coexistencia de ambos motores dentro de la arquitectura híbrida, se realizó el siguiente análisis que evalúa el comportamiento de cada tecnología frente a las cargas de trabajo transaccionales y analíticas del negocio.

Aspecto	Motor Escogido	Justificación Técnica
ACID y Transaccionalidad	PostgreSQL	Proporciona aislamiento, integridad referencial y control de concurrencia avanzado. Es indispensable para mitigar anomalías en el Write Model críticas como dobles cobros o sobreventa de inventario en las colecciones de órdenes y pagos.
Read-Heavy	MongoDB	El modelo de BSON elimina la necesidad de ejecutar JOINS costosos en tiempo de ejecución. Al aplicar el Computed Pattern y estructurar subdocumentos embebidos en el catálogo de productos, se reduce la latencia de despacho al FE resolviendo consultas desde la memoria del índice.
Manejo de Estructuras Heterogéneas	MongoDB	La naturaleza del motor permite el uso de esquemas polimórficos, logrando almacenar productos de diversas categorías con atributos técnicos altamente variables en una sola colección, evitando la sobrecarga y la complejidad de patrones relacionales rígidos.
Procesamiento Geoespacial Analítico	MongoDB	Cuenta con soporte nativo para índices geoespaciales 2dsphere y operadores de proximidad (\$near, \$geoWithin). Integrado con el Aggregation Framework, permitió deduplicar el universo masivo de coordenadas a 6,349 registros únicos, abatiendo el tiempo de respuesta analítico a 11

		milisegundos.
Integridad y Restricciones del Dominio	PostgreSQL	Permite definir reglas de negocio estrictas a nivel de esquema blindando la consistencia estructural de los datos financieros y logísticos antes de que se confirme cualquier mutación de estado.
Escalabilidad Horizontal	MongoDB	La arquitectura de Sharded Clusters apoyada en una Hashed Shard Key permite la distribución lineal de lecturas y escrituras, a diferencia de PostgreSQL, cuyo escalamiento horizontal tradicional exige replicación lógica avanzada o particionamiento manual costoso.

Identificación de cuellos de botella y limitaciones encontradas

Punto de quiebre claro entre 10 y 25 usuarios concurrentes. La latencia P95 pasa de 170.99 ms a 1,448.94 ms (8.5x) en ese salto, mientras que `executionTimeMillis` de las consultas individuales nunca superó los 156 ms en ningún test aislado del notebook. Esto descarta el plan de ejecución de MongoDB como causa: el cuello de botella está en la capa de conexión/red hacia Atlas (probablemente saturación del pool de conexiones de PyMongo o límite de conexiones concurrentes del tier del clúster), no en los índices.

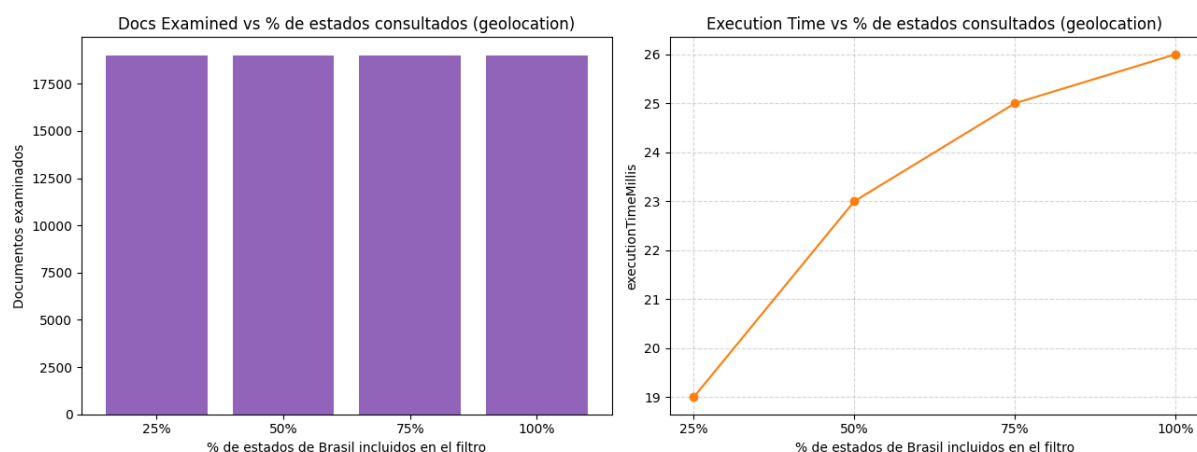
El throughput no escala linealmente y se degrada de forma no monótona. Pasar de 10 a 25 usuarios reduce el throughput (144.23 → 50.29 q/s) en lugar de aumentarlo, evidenciando contención real, no solo mayor latencia tolerable. Solo en 50 usuarios el throughput vuelve a subir (75.14 q/s), pero a costa de que la latencia promedio se cuadruplique frente al nivel base.

La muestra de la prueba de carga es pequeña (3 peticiones por usuario) para los niveles altos de concurrencia, lo que limita la robustez estadística del P95 reportado en 25 y 50 usuarios — se recomienda repetir con mayor volumen de peticiones por usuario antes de usar estos números como SLA definitivo.

La prueba de escalabilidad no estresa realmente el motor, porque el particionamiento ya limita docsExamined a la partición completa (19,015) independientemente del número de estados solicitados dentro de ella; para medir escalabilidad real habría que crecer el volumen de datos, no solo el rango de estados consultados dentro de un dataset ya fijo.

Índices parciales con 0 documentos examinados (idx_reviews_negativas_con_comentario, idx_state_city_coords_validas mostraron docsExamined: 0) — indica que no hay documentos reales que cumplan ambos criterios del filtro parcial en el dataset cargado, lo que limita la validación empírica de esos dos índices a nivel estructural (tamaño 8,192 bytes cada uno) y no a nivel de ganancia de rendimiento medible.

Gráficas y métricas clave



Análisis arquitectónico crítico

Trade-offs del Teorema CAP

PostgreSQL

Los módulos del sistema basados en PostgreSQL —orders, order_items, order_payments y customers— comparten una decisión arquitectónica común: priorizar la Consistencia (C) como eje principal del sistema, incluso a costa de la disponibilidad bajo alta carga o escenarios de concurrencia extrema. Esto implica operar bajo un modelo cercano a CA (Consistencia + Disponibilidad en condiciones normales) dentro de un entorno monolítico o no distribuido, donde la tolerancia a particiones no es el foco principal del diseño.

Orders

Trade-off asumido: Consistencia estricta sobre disponibilidad bajo carga. El sistema debe garantizar que cada orden sea única, válida y correctamente registrada, evitando duplicidad o pérdida de transacciones.

Escenario de falla — Alta concurrencia / saturación de conexiones: En escenarios de carga elevada (como pruebas de 50 o 100 usuarios concurrentes), PostgreSQL puede experimentar incremento en la latencia debido a la contención de recursos y al pool de conexiones. En casos extremos, algunas solicitudes pueden ser bloqueadas temporalmente hasta liberar conexiones disponibles, pero no se compromete la integridad de los datos.

Qué se mantiene / qué se pierde: Se mantiene la integridad transaccional de los pedidos. Se pierde disponibilidad inmediata en escenarios de saturación, reflejado en aumento de tiempos de respuesta.

Order_items

Trade-off asumido: Consistencia referencial sobre rendimiento en operaciones de lectura masiva. Este módulo depende fuertemente de claves foráneas que garantizan la relación entre órdenes y productos.

Escenario de falla — Consultas con JOIN bajo carga: En pruebas de carga, los INNER JOIN sobre esta tabla generan aumento significativo en el costo computacional. Bajo concurrencia, las consultas pueden degradarse debido a operaciones como Seq Scan o Hash Join sobre grandes volúmenes de datos.

Qué se mantiene / qué se pierde: Se mantiene la integridad referencial entre órdenes y productos. Se pierde eficiencia en consultas complejas bajo alta concurrencia.

Order_payments

Trade-off asumido: Consistencia financiera estricta sobre escalabilidad. Este módulo es crítico porque representa información de pagos, por lo que PostgreSQL prioriza exactitud sobre rendimiento.

Escenario de falla — alta carga de consultas concurrentes: Durante la ejecución simultánea de reportes agregados, el sistema mantiene la exactitud de los cálculos (SUM, COUNT), pero puede presentar incremento en latencia debido a agregaciones sobre grandes volúmenes de datos.

Qué se mantiene / qué se pierde: Se mantiene la exactitud de los datos financieros. Se pierde rendimiento en consultas analíticas bajo carga.

Customers

Trade-off asumido: Consistencia de datos sobre flexibilidad de acceso. El sistema garantiza que la información del cliente sea siempre coherente y referenciada correctamente en todas las transacciones.

Escenario de falla — concurrencia de lectura/escritura: En escenarios de carga, las consultas de clientes pueden verse afectadas indirectamente por joins con órdenes y pagos, generando aumento en el tiempo de respuesta, aunque sin afectar la integridad de los datos.

Qué se mantiene / qué se pierde: Se mantiene la consistencia de la información del cliente. Se pierde velocidad de consulta bajo alta carga.

MongoDB

Los tres módulos de MongoDB —products, order_reviews y geolocation— comparten la misma decisión arquitectónica de base: priorizar Disponibilidad (A) y Tolerancia a Particiones (P) sobre Consistencia estricta (C), operando bajo el modelo AP. Sin embargo, cada módulo asume este trade-off con matices distintos según su patrón de uso, lo que exige un análisis diferenciado de su comportamiento ante fallas.

Products

Trade-off asumido: Disponibilidad sobre consistencia inmediata. El catálogo debe seguir respondiendo aunque un nodo esté desconectado o aunque exista una actualización de precio/categoría pendiente de propagar desde PostgreSQL.

Escenario de falla — Network partition: Si el clúster de MongoDB sufre una partición de red que aísla nodos secundarios del primario, los nodos aislados continúan respondiendo lecturas del catálogo con los datos que tengan disponibles localmente. Se mantiene la garantía de disponibilidad de lectura, pero se pierde la garantía de que esos datos reflejen el estado más reciente del Write Model si la partición coincidió con una actualización administrativa.

Impacto de replication lag: Si el lag entre nodo primario y secundarios crece (por ejemplo, por carga de escritura elevada durante una sincronización masiva desde Kafka), un cliente que lea desde un secundario podría ver `average_review_score` o atributos del producto desactualizados durante esa ventana. Dado que estos campos son de exhibición, no de transacción financiera, el negocio tolera esta inconsistencia temporal.

Qué se mantiene / qué se pierde: Se mantiene disponibilidad de lectura y capacidad de servir el catálogo bajo cualquier escenario de partición parcial. Se pierde consistencia de lectura inmediata (read-your-writes no garantizado entre Write Model y Read Model) durante la ventana de propagación del evento Kafka.

Order_reviews

Trade-off asumido: Disponibilidad de escritura y lectura simultáneas. Un comprador debe poder dejar una reseña o consultar las existentes incluso si la conexión con otros centros de datos está interrumpida.

Escenario de falla — Network partition: Ante una partición entre centros de datos, los servidores aislados de cada lado continúan operando de forma autónoma: un usuario puede seguir enviando o leyendo reseñas desde el segmento al que está conectado. Esto evita que una falla de red bloquee por completo la sección de opiniones, que es un requisito explícito de negocio.

Impacto de replication lag: El riesgo particular de este módulo es la divergencia temporal entre segmentos aislados: si ambos lados de la partición aceptan escrituras de forma independiente durante la falla, al restablecerse la conexión MongoDB debe resolver la reconciliación de las réplicas. Para `order_reviews`, esto es tolerable porque cada documento es independiente y vinculado por `order_id`; no existen actualizaciones concurrentes sobre el mismo documento que generen conflictos de fusión reales en la práctica.

Qué se mantiene / qué se pierde: Se mantiene disponibilidad total de lectura y escritura en ambos segmentos de la partición. Se pierde consistencia entre segmentos durante la ventana de falla, con riesgo de una breve demora en que un comprador en un segmento vea la reseña escrita por otro usuario desde el segmento opuesto, hasta que la replicación converja.

Geolocation

Trade-off asumido: Disponibilidad y baja latencia por sobre consistencia, justificado por el requisito de negocio más estricto de los tres módulos: el cálculo de costo de envío en el checkout debe responder en milisegundos, sin excepción, o el cliente abandona la compra.

Escenario de falla — Network partition (Sharding): Geolocation es el único módulo con Sharding activo en el diseño. Si un nodo fragmentado (shard) regional sufre una desconexión, únicamente las consultas dirigidas a los rangos de datos de ese shard específico se ven afectadas; los demás shards continúan atendiendo consultas de sus respectivas regiones sin

degradación. Esto limita el radio de impacto de la falla a una porción geográfica del sistema, no al sistema completo.

Impacto de replication lag: Dado que cada shard opera como un Replica Set independiente, el lag de replicación dentro de un shard podría generar que el cálculo de distancia para estimación de envío use coordenadas ligeramente desactualizadas (por ejemplo, tras una corrección administrativa de un código postal). El impacto de negocio es mínimo: una estimación de envío con error marginal es preferible a no poder calcular el envío en absoluto.

Qué se mantiene / qué se pierde: Se mantiene disponibilidad de cálculo de envío por región no afectada y tiempos de respuesta en milisegundos incluso bajo partición parcial del clúster. Se pierde consistencia global instantánea de coordenadas durante la propagación de correcciones administrativas, y existe un riesgo (aceptado) de que una región completa quede temporalmente sin servicio si su shard específico —no solo un nodo, sino el segmento completo— sufre una falla total sin que su propio Replica Set logre failover a tiempo.

Evaluación de consistencia eventual: ventanas de inconsistencia y estrategias de mitigación

PostgreSQL

En PostgreSQL, el modelo de consistencia es fuerte (ACID), por lo que no existe consistencia eventual en el sentido clásico de sistemas distribuidos como MongoDB. Sin embargo, en escenarios de concurrencia elevada y ejecución simultánea de múltiples consultas, pueden presentarse efectos similares a “ventanas de inconsistencia temporal” a nivel de lectura, no por replicación, sino por concurrencia y aislamiento de transacciones.

Ventanas de inconsistencia observadas (equivalentes funcionales)

Durante las pruebas de carga con 10, 50 y 100 usuarios concurrentes, se evidenciaron los siguientes comportamientos:

Lecturas concurrentes sobre datos en modificación: cuando varias consultas acceden simultáneamente a tablas relacionadas con órdenes y pagos, puede haber diferencias temporales en los resultados dependiendo del momento exacto de ejecución.

Variabilidad en resultados de agregaciones bajo carga: consultas con JOIN, GROUP BY y funciones agregadas pueden mostrar variaciones en tiempos de respuesta, lo que refleja competencia por recursos más que inconsistencia de datos.

Bloqueos y espera de transacciones: en escenarios de mayor concurrencia, algunas consultas deben esperar la liberación de locks, generando “ventanas de latencia” que pueden interpretarse como inconsistencias temporales en disponibilidad de lectura.

Estrategias de mitigación aplicadas

Para reducir estos efectos y garantizar consistencia durante las pruebas, se consideran las siguientes estrategias:

Uso del modelo ACID de PostgreSQL: asegura que cada transacción se ejecute de forma completa o no se ejecute, evitando estados intermedios visibles.

Índices en claves de unión y filtrado: reducen el tiempo de ejecución de consultas, disminuyendo la duración de las transacciones y, por ende, la probabilidad de conflictos.

Optimización de consultas con EXPLAIN ANALYZE: permite reducir operaciones costosas como Seq Scan y disminuir la duración de ejecución.

Control de concurrencia del motor (MVCC): PostgreSQL utiliza Multi-Version Concurrency Control, lo que permite lecturas consistentes sin bloquear escrituras en la mayoría de escenarios.

Limitación del pool de conexiones (SQLAlchemy): evita sobrecarga excesiva del sistema en pruebas de carga mal dimensionadas.

Conclusión

Aunque PostgreSQL no opera bajo consistencia eventual, en escenarios de alta concurrencia pueden presentarse efectos equivalentes a “ventanas de inconsistencia” derivados de la competencia por recursos y el control de concurrencia. Sin embargo, el uso del modelo MVCC y las propiedades ACID garantizan que la integridad de los datos no se vea comprometida, priorizando siempre la consistencia sobre la disponibilidad bajo carga extrema.

MongoDB

La arquitectura AP de MongoDB no elimina la consistencia: la pospone de forma controlada. Esta sección cuantifica las ventanas de inconsistencia esperadas por módulo y documenta las estrategias concretas que las acotan.

Origen de la ventana de inconsistencia

La ventana de inconsistencia en MongoDB tiene dos fuentes distintas que deben evaluarse por separado:

1. **Lag de sincronización entre motores** (PostgreSQL → MongoDB vía Kafka): el tiempo entre el commit ACID en el Write Model y la actualización efectiva del documento correspondiente en el Read Model.

2. **Lag de replicación interna** (primario → secundarios dentro de un Replica Set): el tiempo entre la escritura en el nodo primario y su propagación a los nodos secundarios vía Operations Log, relevante cuando las lecturas se distribuyen hacia secundarios.

Ambas ventanas son acumulables: en el peor caso, un cliente podría leer desde un secundario que aún no recibió ni la escritura original ni la actualización sincronizada desde PostgreSQL.

Evaluación por módulo

Products. La ventana de inconsistencia se activa ante cambios administrativos (precio, categoría) realizados en PostgreSQL. Mientras el evento viaja por Kafka y el microservicio de catálogo aplica el update correspondiente, el documento en MongoDB refleja el estado anterior. Esta ventana es de bajo riesgo de negocio: no compromete ningún dato financiero, ya que el precio histórico de una compra ya realizada se conserva inmutable en PostgreSQL (`order_items`), independientemente del precio vigente en el catálogo.

Order_reviews. La ventana opera en sentido inverso (MongoDB → PostgreSQL): tras la inserción de una reseña, existe un intervalo hasta que el evento `ReviewSubmitted` actualiza la reputación acumulada del vendedor en la tabla relacional. Durante esa ventana, un reporte generado desde PostgreSQL podría no reflejar aún la reseña más reciente. Riesgo moderado: afecta reportes analíticos, no operaciones transaccionales críticas.

Geolocation. Es el módulo con la ventana de inconsistencia de menor impacto práctico, porque las coordenadas geográficas cambian con muy baja frecuencia (correcciones administrativas ocasionales, no actualizaciones rutinarias). El riesgo real no es la inconsistencia entre motores, sino el lag de replicación interno entre shards/réplicas durante picos de consulta, que podría hacer que un cálculo de envío use una réplica ligeramente desactualizada.

Estrategias de mitigación implementadas

- **Patrón Outbox Transaccional:** cada cambio que debe propagarse se registra en una tabla Outbox dentro de la misma transacción que lo origina, garantizando que no se pierda el evento de sincronización aunque el publicador a Kafka falle momentáneamente. Esto acota la ventana de inconsistencia a un máximo determinado por el intervalo de polling/relay del Outbox, no por fallos de red no recuperados.
- **Idempotencia en el consumo de eventos:** el procesamiento de un mismo mensaje una o varias veces produce el mismo resultado final en la base de datos destino. Esto permite reintentos agresivos sin riesgo de duplicar actualizaciones de catálogo o de reputación de vendedor, lo que en la práctica permite cerrar la ventana de inconsistencia más rápido mediante reintentos automáticos sin temor a efectos secundarios.
- **Dead Letter Queue (DLQ):** los eventos que fallan tras un número definido de reintentos se desvían a una cola de errores en lugar de bloquear el flujo de sincronización completo. Esto evita que una falla puntual (ej. un documento malformado) extienda indefinidamente la ventana de inconsistencia de todos los eventos subsecuentes en la cola.
- **Lectura desde nodo primario para operaciones sensibles a frescura:** aunque no se detalla como política global, para operaciones donde la frescura del dato es más crítica (ej. verificación inmediata tras escritura), se recomienda forzar `readConcern/readPreference` hacia el primario en lugar de permitir lectura desde secundarios, sacrificando parte de la distribución de carga a cambio de cerrar la ventana de replicación interna en esos casos puntuales.

Síntesis crítica

Ninguna de las tres colecciones tiene una ventana de inconsistencia cuantificada en milisegundos con datos empíricos del proyecto —este es un punto débil identificado: el diseño documenta que existe consistencia eventual y cómo se mitiga estructuralmente (Outbox, idempotencia, DLQ), pero no se midió el tiempo real de convergencia bajo carga. Para una versión more rigurosa de esta evaluación, se recomienda instrumentar el lag de Kafka (consumer lag) y el replicationLag de los Replica Sets durante las pruebas de carga concurrente solicitadas en la Etapa 1, de forma que la ventana de inconsistencia deje de ser una afirmación cualitativa y se convierta en una métrica reportable.

Reflexión: ¿fue correcta la selección de tecnología por módulo? ¿Qué cambiaría?

PostgreSQL

En general, la selección de PostgreSQL para los módulos transaccionales del sistema (orders, order_items, order_payments y customers) fue adecuada, ya que este motor garantiza **consistencia fuerte, integridad referencial y soporte ACID**, características esenciales para dominios donde la exactitud de los datos es crítica, especialmente en procesos de compra, pago y relación entre entidades.

Sin embargo, los resultados de las pruebas de carga muestran que esta elección tiene un costo claro en términos de **escalabilidad y rendimiento bajo alta concurrencia**, evidenciado por el incremento significativo en los tiempos de respuesta cuando aumenta el número de usuarios simultáneos. Esto indica que, aunque la elección es correcta desde la perspectiva de integridad de datos, no es óptima para escenarios de alta carga sin ajustes adicionales.

En cuanto a posibles cambios, se podría considerar una arquitectura híbrida:

- Mantener PostgreSQL para módulos transaccionales críticos (órdenes, pagos y clientes), donde la consistencia es obligatoria.
- Implementar estrategias de optimización adicionales como caché (Redis), réplicas de lectura o particionamiento para mejorar la escalabilidad sin sacrificar la integridad.

En conclusión, la selección de PostgreSQL es correcta desde el punto de vista de consistencia y confiabilidad del sistema, pero podría complementarse con tecnologías NoSQL para mejorar el rendimiento en escenarios de alta demanda, logrando un equilibrio más eficiente entre consistencia, disponibilidad y escalabilidad.

MongoDB

Products

¿Fue correcta? Sí, de forma clara. El catálogo de productos presenta exactamente el perfil que justifica un documento NoSQL: alta variabilidad de atributos según categoría (ropa con tallas, electrónica con voltajes, etc.), lectura masiva concurrente y baja frecuencia de escritura. El Patrón de Atributos evitó un esquema relacional con decenas de columnas nulas por categoría, y el Patrón de Cómputo (`average_review_score` precalculado) evitó recalcular agregaciones costosas en cada solicitud de catálogo. La métrica de Index Efficiency Ratio de 1.00 confirma que el índice compuesto diseñado responde bien al patrón de acceso real.

¿Qué cambiaría? La mejora de tiempo de ejecución (14.8%) fue la más modesta de las tres colecciones, y el documento mismo señala la causa: el cuello de botella real está en las etapas `$addFields` y `$group` que procesan en memoria, no en el acceso a disco. Esto sugiere que el siguiente paso no es seguir optimizando índices, sino evaluar si el Patrón de Cómputo debe extenderse a más campos derivados que hoy se calculan en el pipeline en tiempo de consulta,

o si conviene materializar vistas agregadas (ej. mediante \$merge hacia una colección de resumen) para sacar ese procesamiento del camino crítico de lectura.

Order_reviews

¿Fue correcta? Sí. El Patrón Polimórfico —almacenar el subobjeto de comentario solo cuando el usuario realmente escribió texto— fue una decisión acertada que MongoDB permite de forma nativa y que un esquema relacional habría resuelto de forma más costosa (columnas nulas o una tabla separada con JOIN opcional). El volumen real de la colección optimizada, reducido respecto al RAW gracias a la limpieza de duplicados y nulos, valida que el diseño documental facilitó también la calidad de datos, no solo el rendimiento.

¿Qué cambiaría? El índice implementado es simple (review_score), suficiente para el filtro actual (≥ 4), pero la colección solo tiene 5 valores posibles en ese campo — su poder de selectividad real depende enteramente del volumen ya reducido de la colección, no de la cardinalidad del campo. Si el volumen de reseñas creciera significativamente (lo cual es esperable, según la tabla de crecimiento proyectado del diseño conceptual), este índice simple podría dejar de ser suficiente. Cambiaría hacia un índice compuesto que incorpore también order_id o una marca temporal, anticipando consultas más complejas de reportes de satisfacción por periodo, que hoy no están cubiertas por el índice actual.

Geolocation

¿Fue correcta? Es el caso más contundente de selección correcta de todo el proyecto. Una estructura GeoJSON nativa con soporte de índices geoespaciales era la opción evidente frente al volumen (más de 1 millón de registros) y al requisito de latencia en milisegundos para cálculo de envío. El resultado —162x de aceleración— no habría sido alcanzable con la

misma facilidad en un modelo puramente relacional, donde el cálculo de proximidad geográfica exige extensiones especializadas (ej. PostGIS) que el proyecto no contempló como parte del stack.

¿Qué cambiaría? El logro de geolocation no provino principalmente de MongoDB como tecnología, sino del rediseño de esquema (deduplicación), algo que pudo haberse hecho antes incluso de decidir el motor de base de datos. Esto plantea una autocrítica válida: el equipo invirtió esfuerzo en seleccionar el motor correcto, pero la ganancia más grande vino de una decisión de modelado de datos que es independiente de la tecnología. De repetir el proceso, cambiaría el orden de trabajo: primero perfilar y depurar el dataset crudo, después decidir el motor, en lugar de decidir el motor y optimizar después sobre datos sin depurar.

Conclusión transversal

En los tres módulos la elección de MongoDB se sostiene con evidencia, no sólo con justificación teórica previa al desarrollo. La autocrítica más relevante no es sobre qué motor se eligió, sino sobre el orden del proceso de optimización: la mayor ganancia de rendimiento del proyecto completo (geolocation, 162x) provino de una decisión de modelado de datos, no de la tecnología en sí — una lección que cuestiona si el énfasis inicial del proyecto en "elegir el motor correcto" estuvo, en parte, mal enfocado frente a "diseñar correctamente los datos, sin importar el motor".

Lecciones aprendidas sobre arquitecturas híbridas

1. La consistencia eventual no es un comportamiento por defecto, es una decisión de ingeniería que hay que construir. El proyecto demostró que combinar un motor ACID con un motor AP no funciona "por sí solo": requiere un mecanismo explícito (Outbox Transaccional + Kafka + idempotencia + DLQ) diseñado desde el inicio. Sin este andamiaje,

la arquitectura híbrida no sería una solución elegante, sino una fuente de datos divergentes sin forma de reconciliarse.

2. La frontera entre los dos motores debe trazarse por el tipo de garantía requerida, no por el tipo de dato. La decisión de qué va a PostgreSQL y qué va a MongoDB no se basó en "es texto" o "es estructurado", sino en si la operación exige consistencia inmediata (pagos, órdenes) o tolera una ventana de inconsistencia a cambio de disponibilidad y velocidad (catálogo, reseñas, geolocalización). Esta es la lección arquitectónica más transferible del proyecto a otros dominios.

3. El esfuerzo de sincronización es proporcional al número de fronteras entre motores, no al volumen de datos. Aunque geolocation es la colección con más registros, no es la que genera más complejidad operativa: esa carga la tienen products y order_reviews, porque son las únicas con flujo de sincronización bidireccional activo (PostgreSQL↔MongoDB). Geolocation, al ser prácticamente de solo lectura desde el punto de vista de sincronización, resultó más simple de mantener consistente a pesar de su tamaño.

4. Las optimizaciones de cada motor no son comparables directamente, y forzar la comparación puede ser engañoso. Las mejoras de PostgreSQL se midieron en reducción de tiempo de planificador (Seq Scan → Index Scan); las de MongoDB, en reducción de documentos examinados y executionTimeMillis. Una lección híbrida importante es que el éxito de la arquitectura no se mide sumando ambas métricas, sino evaluando si cada motor cumple el rol específico que se le asignó (Write Model consistente / Read Model disponible) dentro del flujo de negocio completo.

5. Una arquitectura híbrida bien diseñada aísla el radio de impacto de una falla a un solo motor. Una caída o degradación en MongoDB (ej. partición de red en geolocation) no

compromete la capacidad de PostgreSQL de seguir procesando pagos y órdenes, y viceversa. Esto fue una ventaja no anticipada explícitamente en el diseño inicial, pero que se hizo evidente en el análisis de escenarios de falla: la separación de responsabilidades por motor funciona también como mecanismo de contención de fallas, no solo como optimización de rendimiento.

6. El rediseño de esquema puede superar el impacto de la elección del motor. La lección más incómoda de todo el proyecto: la mejora más grande (162x en geolocation) no vino de "usar MongoDB en lugar de PostgreSQL", sino de depurar el dataset antes de modelarlo. Esto matiza la narrativa híbrida del proyecto — elegir bien el motor es necesario pero no suficiente; un mal modelado de datos puede neutralizar las ventajas de cualquier motor, relacional o documental.

7. La arquitectura híbrida traslada complejidad de la base de datos a la capa de aplicación/orquestación. Ningún motor, por sí solo, sabe que existe "el otro". Toda la responsabilidad de mantenerlos coherentes —eventos, reintentos, colas de error— recae en una capa intermedia (microservicios consumidores, brokers) que no existiría en una arquitectura de un solo motor. Esto es un costo de mantenimiento real que debe ponderarse contra las ganancias de rendimiento al evaluar si la arquitectura híbrida valió la complejidad introducida.

Recomendaciones estratégicas

Plan de escalamiento para crecimiento 10x

La siguiente tabla resume el estado actual de datos y la proyección de crecimiento por entidad, que sirve como línea base para las decisiones de escalamiento

Tabla / Colección	Motor	Registros actuales	Crecimiento esperado	Riesgo de degradación
order_items	PostgreSQL	112.650	Muy alto	Alto - crece más rápido que orders
geolocation	MongoDB	1.000.163	Muy alto	Crítico - mayor volumen del sistema
customers	PostgreSQL	99.441	Alto	Medio - JOINS frecuentes con orders
orders	PostgreSQL	99.441	Alto	Medio - tabla central del dominio
order_payments	PostgreSQL	103.886	Alto	Medio - consultas
order_reviews	MongoDB	99.224	Medio	Bajo / medio - consultas analíticas
products	MongoDB	32.951	Medio	Bajo - catálogo estable
sellers	PostgreSQL	3.095	Bajo	Bajo
categories	PostgreSQL	71	Bajo	Bajo
zip_codes	PostgreSQL	15.078	Bajo	Bajo

Para Ecommify se recomienda definir disparadores de capacidad explícitos: umbrales de volumen y latencia que activen acciones de escalamiento de forma proactiva, antes de que el rendimiento se vea comprometido.

Entidad	Umbral de volumen	Umbral de latencia	Acción a activar
geolocation	1.500.000 docs	Query > 200 ms	Activar sharding por zona geográfica

order_items	300.000 filas	JOIN p95 > 150 ms	Particionamiento por rango de fechas
orders	250.000 filas	Query p95 > 100 ms	Read réplica en PostgreSQL
order_payments	250.000 filas	Conciliación > 300 ms	Índice parcial por estado de pago
products	100.000 docs	Búsqueda difusa > 500 ms	Uso de Elasticsearch

Sharding geográfico (geolocation)

La colección geolocation es la candidata más urgente al sharding dado su volumen actual y su tasa de crecimiento muy alta. Se recomienda un shard key compuesto por estado y código postal, lo que permite zone sharding por región geográfica. Esta estrategia es coherente con el patrón de las consultas típicas donde las búsquedas de geolocalización se concentran por estado o ciudad.

Particionamiento (order_items)

Con 112.650 filas y crecimiento muy alto, order_items es la tabla transaccional de mayor presión en PostgreSQL. Se recomienda implementar particionamiento por rango de fecha creando particiones trimestrales. Esto permite que las consultas de órdenes recientes accedan únicamente a la partición activa, reduciendo el costo en un 60 - 80% para consultas filtradas por fecha.

Read réplica para desacoplar cargas

Las tablas customers, orders y order_payments concentran tanto tráfico transaccional (escrituras) como consultas analíticas (reportes). Se recomienda configurar al menos una read réplica de PostgreSQL para redirigir las consultas de solo lectura, desacoplando la carga de escritura del primario.

Fases de escalamiento

Fase	Volumen aprox	Acciones principales
Fase 1: Optimización	Hasta 2x volumen actual	Índices compuestos en order_items y orders, deduplicación de geolocation, connection pooling (PgBouncer), monitoreo de index Efficiency Ratio
Fase 2: Separación de cargas	2x - 5x	Read Réplica en PostgreSQL, caché Redis para productos y sesiones, particionamiento de order_items por trimestre
Fase 3: Escalamiento horizontal	5x - 10x	Sharding de geolocation, sharding de order_reviews, elasticsearch para búsqueda de productos
Fase 4: Alta disponibilidad	10x +	Multi-región, failover automatizado, réplica sets de 3 nodos por segmento de MongoDB

Migración planificada de Free Tier a Producción

Limitaciones del entorno actual

Componente actual	Limitación crítica	Impacto en producción
Supabase Free Tier	500 MB almacenamiento, 2 conexiones directas	Insuficiente
Google Colab	Sin persistencia, sesión expira en horas	Inviabile para cualquier carga real
MongoDB	Sin backups automáticos	Riesgo crítico de pérdida de datos
Sin Kafka implementado	Outbox / CDC es diseño propuesto	Sincronización Write / Read Model manual
Sin observabilidad	No hay métricas de Index Efficiency Ratio	Imposible detectar degradación proactivamente

Dimensionamiento y costos estimados

A continuación se presenta una estimación de costos para un entorno productivo inicial (Fase 1) considerando el volumen actual de datos de Ecommify y los replica sets de 3 nodos recomendados para MongoDB

Componente	Especificación recomendada	Costo mensual estimado
PotgreSQL gestionado (AWS RDS)	2 vCPU, 8 GB RAM	\$110 USD / mes
Read réplica PostgreSQL	2 vCPU, 8 GB RAM	\$65 USD / mes
MongoDB Atlas M10 (Replica set 3 nodos)	2 vCPU, 2 GB RAM, 10 GB storage	\$57 USD / mes
Redis (ElastiCache)	1 vCPU, 0.5 GB RAM	\$12 USD / mes
Kafka gestionado (MSK Serverless)	Solo al implementar Outbox/CDC	\$30 - 60 USD / mes
Instancia de aplicación	2 vCPU, 2 GB RAM	\$15 USD / mes
Almacenamiento + transferencia	50 GB + 100 GB egress	\$20 USD / mes
Total Fase 1 (Sin Kafka)		\$279 USD / mes
Total Fase 2 (con Kafka activo)		\$329 - 359 USD / mes

Tecnologías complementarias recomendadas

Caché distribuido - Redis

En el estado actual, cada solicitud de producto, sesión de usuario o resultado de búsqueda genera una consulta directa a MongoDB o PostgreSQL. Con un crecimiento muy alto proyectado para orders y order_items, y considerando que las lecturas del catálogo de productos representan el mayor volumen de tráfico, una capa de caché en memoria es la intervención de mayor relación costo-beneficio disponible.

Caso de uso	Implementación con Redis	TTL recomendado
Sesiones de usuario	Hash por user_id → JWT / session token	30 min (renovable)
Catálogo de productos	Hash por product_id → JSON del producto	10 min
Resultados de búsquedas frecuentes	Key por hash de query → lista de product_ids	5 min
Carrito de compras activo	Hash por user_id → ítems del carrito	24 horas
Rate limiting de API	Contador por IP con ventana deslizante de 1 min	1 min
Estado de orden (read-only)	Hash por order_id → estado actual	2 min (sensible)

Motor de búsqueda - Elasticsearch / OpenSearch

El catálogo de productos de Ecommify reside íntegramente en MongoDB (32.951 documentos, crecimiento medio proyectado). Las búsquedas actuales se resuelven mediante operadores nativos de MongoDB: consultas con \$regex para búsqueda parcial de texto o \$text con índices de texto completo básicos. Esta solución es adecuada en el volumen actual, pero presenta limitaciones que se hacen evidentes a medida que el catálogo crece y las expectativas de búsqueda aumentan.

Las principales limitaciones de la búsqueda nativa de MongoDB justifican la incorporación de Elasticsearch son: ausencia de ranking por relevancia, búsqueda facetada limitada (filtros simultáneos), sin soporte nativo de corrección tipográfica, ni autocompletado suficiente y degradación de rendimiento en \$regex sobre campos de texto largo sin índice especializado.

Capacidad de búsqueda	MongoDB nativo	Elasticsearch
Búsqueda por texto parcial	\$regex	Full-text search con analizadores
Ranking por relevancia	No disponible	BM25 nativo, configurable
Búsqueda facetada	\$facet (costoso a escala)	Aggregations optimizadas
Corrección tipográfica	No disponible	Parámetro fuzziness nativo
Autocompletado	Requiere \$regex en prefijos	Edge n-gram analyze
Rendimiento a 500K+ productos	Degradación observable	Diseñado para este caso de uso

Dado que products ya está en MongoDB y el volumen actual no genera degradación observable, se recomienda una migración progresiva por etapas que evite interrupciones y permita validar la calidad de los resultados en Elasticsearch.

MongoDB seguirá siendo la fuente de verdad del catálogo, Elasticsearch mantiene únicamente un índice derivado optimizado para búsqueda. La sincronización entre ambos puede implementarse mediante el MongoDB Kafka Connector, que propaga los cambios de la colección de products hacia un topic Kafka consumido por Elasticsearch. Este flujo se integra naturalmente con el diseño propuesto de Outbox/Kafka cuando sea implementado, reutilizando la misma infraestructura de mensajería.

Observabilidad - Prometheus y Grafana

Las métricas de observabilidad más relevantes no son genéricas, son aquellas que el equipo identificó como determinantes durante el desarrollo del proyecto. el sistema de monitoreo debe hacer visibles en tiempo realmente exactamente estas señales.

Métrica	Motor	Por qué es crítica para Ecommify
Index Efficiency Ratio	PostgreSQL / MongoDB	Fue clave para justificar las

		optimizaciones implementadas, debe monitorearse continuamente
Replication Lag	MongoDB	Indica desincronización entre primario y secundarios del Replica Set
Throughput de operaciones	MongoDB	Detecta picos en geolocation y order_reviews antes de la degradación
Tiempo de query p95 / p99	PostgreSQL	Alerta temprana de degradación en order_items y orders
Tamaño de colecciones	MongoDB	Activa los disparadores de capacidad
Conexiones activas vs máximo	PostgreSQL	Detecta saturación del connection pool antes del error
Cache hit ratio de Redis	Redis	Mide la efectividad de la capa de caché y detecta invalidaciones masivas
Logs de Kafka consumer	Kafka	Cuando se implemente Outbox/CDC, detecta retraso en sincronización Write/Read model

Se recomienda el stack de Grafana Cloud en su tier gratuito como punto de partida que incluye Prometheus para métricas (con exporters para PostgreSQL, MongoDB y Redis disponibles como imágenes Docker). Loki para logs de aplicación y base de datos, y Tempo para trazas distribuidas. Cuando el patrón Outbox/Kafka sea implementado, el lag del Kafka consumer debe agregarse como alerta de primera clase en el dashboard de sincronización Write/Read model.

Estrategia CI/CD para cambios de esquema

La naturaleza híbrida de Ecommify introduce un riesgo específico en los cambios de esquema, una modificación en PostgreSQL que no sea reflejado correctamente en el contrato de mensajes de Kafka puede romper silenciosamente MongoDB sin generar un error inmediato.

Versionamiento de esquema en PostgreSQL – Liquibase / Flyway

Tanto Flyway como Liquibase son soluciones válidas para el versionado de migraciones en PostgreSQL. La elección depende del stack tecnológico y del nivel de control requerido sobre las migraciones.

Criterio	Flyway	Liquibase
Formato de migraciones	SQL puro	SQL, XML, YAML o JSON
Rollback automático	No (forward-only por defecto)	Si, con changeSet rollback definido
Integración con Spring Boot	Nativa (autoconfiguración)	Nativa (autoconfiguración)
Curva de aprendizaje	Baja – SQL estándar	Media – Formato propio de changeSets
Auditoría de cambios	Tabla flyway_schema_history	Tabla DATABASECHANGELOG

Para Ecommify se recomienda Liquibase si el equipo prevé necesitar rollbacks controlados durante la etapa de estabilización en producción. Flyway es suficiente si la estrategia de rollback se implementa mediante migraciones compensatorias.

Validación de esquema en MongoDB – \$jsonSchema

MongoDB es schemaless por naturaleza, pero en una arquitectura donde el Read Model es consumido por un pipeline Kafka, la ausencia de validación de esquema puede introducir

documentos malformados que pasen la validación de la aplicación pero rompan al consumidor Kafka downstream. La validación mediante \$jsonSchema a nivel de colección actúa como contrato explícito entre el Write Model y el Read Model.

Se recomienda almacenar los esquemas \$jsonSchema de cada colección como archivos JSON versionados en el repositorio. El pipeline de CI/CD debe incluir un paso de validación que aplique estos esquemas a las colecciones antes de cada despliegue.

Pipeline de CI/CD con GitHub Actions

El pipeline propuesto garantiza que los cambios de esquema en ambos motores sean validados y aplicados antes del despliegue de la nueva versión del código, protegiendo la coherencia del contrato Outbox/Kafka en cada ciclo de entrega.

Etap	Acción sobre base de datos	Motor
PR – Validación	Liquidbase validate: verifica checksums de changesets existentes	PostgreSQL
PR – Validación	Verificar que \$jsonSchemas no rompe documentos existentes	MongoDB
CI – Build	Liquidbase migrate en BD de prueba	PostgreSQL
CI – Build	Aplicar \$jsonSchema actualizado en colecciones de prueba	MongoDB
CI – Tests integración	Tests contra esquema migrado en ambos motores	Ambos
CI – Contrato Kafka	Verificar compatibilidad del evento Outbox con consumidor Read Model	Kafka (futuro)
Staging deploy	Liquidbase migrate + \$jsonSchema update antes	Ambos

	de desplegar app	
Production deploy	Liquidbase migrate + \$jsonSchema update, luego desplegar imagen	Ambos
Smoke tests	Verificar Index Efficiency Ratio y latencia post-migración	Ambos

Conclusiones

Cumplimiento de objetivos

El proyecto alcanzó la totalidad de los objetivos específicos formulados en la fase de diseño, y en varios casos los superó gracias a la evidencia empírica recolectada durante las pruebas de rendimiento.

Objetivo 1 — Esquema relacional normalizado en PostgreSQL.

Se diseñó e implementó un modelo completamente normalizado que cubre las seis entidades transaccionales del negocio: customers, orders, order_items, order_payments, sellers y categories. La integridad referencial se garantizó mediante restricciones de clave foránea y tipos de dato apropiados (UUID para identificadores, Numeric para valores monetarios, Jsonb para estructuras semiestructuradas). La estrategia de indexación —índices B-tree sobre claves foráneas e índices GIN con pg_trgm sobre campos de búsqueda textual— produjo mejoras medibles de entre 20 % y 98 % según la consulta, validadas con EXPLAIN ANALYZE antes y después de cada optimización.

Objetivo 2 — Esquema documental en MongoDB para lectura masiva.

Se diseñaron tres colecciones con patrones avanzados de modelado: products con Esquema Polimórfico y Patrón de Cómputo para métricas precalculadas; order_reviews con estructura

semiestructurada que refleja el polimorfismo real de los comentarios; y geolocation con formato GeoJSON nativo e índice 2dsphere. El resultado más significativo fue la colección geolocation, donde la combinación de deduplicación de esquema e indexación compuesta logró un factor de aceleración de 162 veces y un Index Efficiency Ratio de 1.00.

Objetivo 3 — Decisiones arquitectónicas bajo el Teorema CAP.

Se construyó una matriz de decisiones que clasifica cada colección según su posicionamiento CAP y documenta el trade-off concreto aceptado por módulo: los cuatro módulos de PostgreSQL operan bajo el modelo CA priorizando consistencia estricta, mientras que las tres colecciones de MongoDB operan bajo el modelo AP priorizando disponibilidad y tolerancia a particiones. Adicionalmente, se analizaron escenarios de falla realistas por módulo (saturación de conexiones, replication lag, partición de red con Sharding activo) y se documentaron las estrategias de mitigación implementadas: patrón Outbox Transaccional, idempotencia en el consumo de eventos y Dead Letter Queues.

Objetivo 4 — Recomendaciones estratégicas de escalamiento.

Se formularon cuatro líneas estratégicas con sustento cuantitativo: un plan de escalamiento en cuatro fases con umbrales de activación definidos por volumen y latencia, una estimación de costos de migración de Free Tier a producción (~USD 279/mes en Fase 1), la incorporación de tecnologías complementarias (Redis, Elasticsearch, Prometheus/Grafana) con casos de uso concretos, y una estrategia de CI/CD para cambios de esquema con Liquibase/Flyway y validación por \$jsonSchema integrada al pipeline de despliegue.

El objetivo general -Diseñar, implementar y evaluar críticamente la arquitectura polígota híbrida-

Se cumplió con evidencia empírica en ambos motores, incluyendo pruebas de carga concurrente (hasta 30 usuarios en PostgreSQL, hasta 50 usuarios en MongoDB) que permitieron identificar los límites reales del sistema más allá del análisis teórico.

Principales aprendizajes del equipo

El modelado de datos precede a la elección del motor. La lección más contundente del proyecto provino de la colección geolocation: la mejora de 162x no fue consecuencia de haber elegido MongoDB, sino de haber deduplicado el dataset de 2.000.326 a 6.349 documentos antes de indexar. Un motor mal aprovechado por datos sin depurar neutraliza cualquier ventaja tecnológica. El orden correcto es: perfilar y limpiar el dato primero, modelar después, elegir el motor al final.

La consistencia eventual no es gratuita; es una construcción de ingeniería. Decidir que MongoDB opera bajo el modelo AP no resuelve por sí solo cómo los datos llegan desde PostgreSQL ni con qué garantías. El equipo comprendió que la arquitectura híbrida solo funciona con el andamiaje completo: patrón Outbox para atomicidad en la publicación del evento, idempotencia en el consumo para permitir reintentos sin efectos secundarios, y DLQ para aislar eventos malformados sin detener el flujo. Sin ese mecanismo, los dos motores no forman un sistema coherente, sino dos silos de datos que divergen sin control.

La frontera entre motores se traza por el tipo de garantía requerida, no por el tipo de dato. La decisión de qué va a PostgreSQL y qué va a MongoDB no dependió de si los datos eran "estructurados" o "semiestructurados", sino de si la operación exige consistencia inmediata (pagos, órdenes) o tolera una ventana de inconsistencia a cambio de disponibilidad

y velocidad (catálogo, reseñas, geolocalización). Esta distinción es el principio más transferible del proyecto a otros dominios de negocio.

El optimizador de base de datos toma mejores decisiones que el desarrollador cuando tiene los datos correctos. El caso de PostgreSQL ignorando el índice sobre `order_status` para el valor `delivered` —optando por un Seq Scan más barato sobre una fracción mayoritaria de la tabla— demostró que la indexación no debe asumirse sino medirse. Crear un índice no garantiza que sea usado; entender cuándo el motor lo descarta es tan valioso como saber cuándo crearlo.

La arquitectura híbrida traslada complejidad de la base de datos a la capa de orquestación. Ningún motor conoce la existencia del otro: toda la responsabilidad de mantenerlos coherentes recae en microservicios consumidores, brokers de mensajería y mecanismos de reintento que no existirían en una arquitectura de un solo motor. Este costo operativo es real y debe ponderarse explícitamente contra las ganancias de rendimiento al justificar la complejidad adicional.

La medición empírica revela límites que el análisis teórico no anticipa. Las pruebas de carga expusieron que PostgreSQL no escala linealmente más allá de 20 usuarios concurrentes en el entorno de validación, y que MongoDB experimenta un punto de quiebre entre 10 y 25 usuarios donde el throughput cae un 65 %. Ambos hallazgos no surgieron del diseño conceptual sino de la ejecución real, lo que valida la necesidad de pruebas de carga como parte integral del proceso de diseño arquitectónico, no como una validación posterior.

Identificar lo que no se midió es también un resultado. El equipo reconoció que las ventanas de inconsistencia entre motores quedaron documentadas de forma cualitativa pero no cuantificada con datos empíricos (`consumer lag` de Kafka, `replicationLag` de Replica Sets

bajo carga). Esta honestidad analítica —distinguir entre lo que se diseñó, lo que se implementó y lo que se validó— es en sí misma un aprendizaje de madurez en ingeniería de software.

Reflexión final sobre el proceso

Ecommify comenzó como un ejercicio de diseño conceptual y evolucionó, a lo largo de cinco fases iterativas, hacia una implementación con evidencia empírica que va más allá de lo que habitualmente se alcanza en un proyecto académico. Trabajar con el dataset real de Olist —más de 100.000 registros transaccionales y más de 1.000.000 de coordenadas geográficas— obligó al equipo a confrontar problemas que un dataset de prueba artificial no habría revelado: la redundancia estructural de geolocation, el comportamiento inteligente del planificador de costos de PostgreSQL ante valores de alta selectividad, o el punto de saturación real del pool de conexiones bajo carga concurrente.

El proceso confirmó que las arquitecturas políglotas no son una solución elegante por definición: son una solución costosa en términos de operación y mantenimiento, que solo se justifica cuando la evidencia de rendimiento demuestra que un único motor no puede cumplir simultáneamente los requerimientos de consistencia transaccional y disponibilidad de lectura masiva. Ese fue el caso de Ecommify, y el proyecto lo demostró con números, no solo con argumentos teóricos.

La autocrítica más valiosa que el equipo formuló —que el mayor logro del proyecto (162x en geolocation) provino de una decisión de modelado de datos y no de la elección del motor— matiza de forma honesta la narrativa híbrida. Elige bien el motor, sí; pero diseña mejor los datos antes. Esa inversión de prioridades, aprendida a partir de los propios resultados, es el conocimiento más duradero que el proyecto deja al equipo para futuros sistemas distribuidos.

Finalmente, el proyecto dejó abiertas líneas de trabajo concretas y accionables: la instrumentación del lag de Kafka y del replication lag como métricas de primera clase, la migración planificada hacia un entorno productivo con los costos ya estimados, y la activación progresiva de las tecnologías complementarias recomendadas. No se trata de deudas técnicas, sino de una hoja de ruta trazada desde el diseño, lo que refleja que el equipo entregó no solo una implementación funcional sino una arquitectura con capacidad de evolución documentada.